

# Optimizing Assertions in Semi-Formal Assertion-Based Verification

Danny Sarraf

Master in Electrical Engineering

Department of Electrical and
Computer Engineering

McGill University

Montreal, Quebec

March 7, 2013

A thesis submitted to McGill University in partial fulfillment
of the requirements of the degree of Master of Engineering

Copyright © Danny Sarraf 2012

## ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my thesis supervisor Dr. Zeljko Zilic for his support, encouragement and inspiration needed to successfully undertake this masters thesis. Throughout my master, he provided me with a clear vision, sound advice and his presence was always there when I needed it. This thesis would not have been possible without his support.

I would like to thank all of my colleagues in the MACS lab for providing a friendly and stimulating environment. In particular, I want to thank Jason Tong for providing me with assistance and making my work a lot easier. I had the pleasure of writing my first journal paper with him during which I was introduced to LaTeX.

I also want to thank Marc Boulé, Omid Sarbishei, Mohammadhossein Neishabouri, Bojan Mihajlovic and others for their kind assistance with writing papers, giving advice and helping in various ways.

During my thesis I had the opportunity of working on a joint NSERC engage project with PMC-Sierra. The colleagues at PMC-Sierra very kind, welcoming and supportive. I would like to extend my thanks Ken Wagner, Patrick Tsinany, Dino Pozzebon, Francois Lambert, Sébastien Regimbal, Andras de Koos and many others... Also, I would like to thank Claude Beauregard, the IFV specialist from Cadence Design Systems who visited PMC-Sierra frequently and provided us with support on IFV.

Prior to my thesis, I had the chance of working on an internship at Gennum Corporations in Ottawa. I learned a lot about Verification at Gennum and many of

the colleagues were very helpful and aided me in my master application. Thanks to Julius Yam, Kishor Mistry and Cristian Samoila.

I would like to thank the Department of Electrical and Computer Engineering at McGill University which has provided me with much needed support and advice.

Finally, I would like to thank my parents and friends for supporting me throughout all my studies at McGill University.

Danny Sarraf

## ABSTRACT

Assertion-based verification (ABV) is a powerful verification approach that has been proven to help digital IC architects, designers, and verification engineers improve design quality and reduce time to market [6].

Assertions are a very powerful, concise and precise mechanism to specify properties of logic design. They run in all verification environments: simulation, emulation/acceleration and formal. In addition, they can be used as checkers or alternatively as assumptions [22].

Some of the major challenges of ABV are that assertions are time-consuming to debug, there's no good way to measure the quality of hand-generated assertions and it's unclear how many assertions one needs to write [32].

This thesis is an attempt at providing new ideas on how to resolve such issues. Different methods of adding assertions to an existing set have been explored and the task of writing enough assertions will be broken down into smaller and more manageable chunks. In order to do so, various types of assertions have first been identified in terms of their usefulness from the designers' and verification engineers' points of view.

Lastly, the implementation of logic synthesis techniques to optimize assertions has been explored. Many significant improvements in terms of minimizing the total number of assertions as well as producing more efficient assertions have been observed.

# ABRÉGÉ

La vérification basée sur l'assertion (Assertion-based Verification - ABV) est une approche de vérification puissante ayant été prouvée à aider les architectes numériques de circuits intégrés, les concepteurs et les ingénieurs de vérification à améliorer la qualité de conception et de réduire le temps de mise sur le marché [6].

Les assertions sont un outil très puissant et précis pour définir les propriétés de la conception logique. Elles peuvent être employées dans tous les environnements de vérification: simulation, émulation ou formelle. Aussi important, elles peuvent être utilisées comme corrections ou encore comme hypothèses [22].

Les inconvénients majeurs d'ABV sont que les assertions nécessitent beaucoup de temps à déboguer, qu'il n'y a pas de bonne façon de mesurer la qualité des assertions produites manuellement et qu'on ne sait pas comment déterminer combien d'assertiosn on a besoin d'écrire [32].

Cette thèse tente d'apporter de nouvelles idées afin de résoudre ces questions. Différentes méthodes d'ajouter des assertions ont été explorées. La tâche de la rédaction d'assez d'assertions a été décomposée en plus petits et plus gérables morceaux. Pour réaliser ceci, différents types d'assertions ont été d'abord identifiés en termes de leur utilité des points de vue des concepteurs et ingénieurs de vérification.

Enfin, la mise en œuvre des techniques de synthèse logique pour optimiser les assertions ont été explorées. Des améliorations notables en termes de minimisation du nombre total d'assertions ainsi que la production d'assertions plus efficaces ont

été observées.

TABLE OF CONTENTS

ACKNOWLEDGMENTS . . . . . ii

ABSTRACT . . . . . iv

ABRÉGÉ . . . . . v

LIST OF TABLES . . . . . x

LIST OF FIGURES . . . . . xiii

1 Introduction . . . . . 1

1.1 Hardware verification . . . . . 1
1.2 Research Collaboration with PMC-Sierra . . . . . 2
1.3 Purpose and Organization of the Thesis . . . . . 3

2 Background on Assertion Based Verification . . . . . 4

2.1 Background on Assertions . . . . . 4
2.1.1 What is an Assertion? . . . . . 4
2.1.2 Evolution and Maturity of ABV . . . . . 5
2.1.3 Advantages of Assertions . . . . . 5
2.1.4 Types of Assertions Written in Verification . . . . . 6
2.2 Extending Assertions . . . . . 6
2.2.1 Extending Assertions Spatially . . . . . 7
2.2.2 Extending Assertions Temporally . . . . . 8
2.2.3 Extending Assertions Combinatorially . . . . . 8
2.2.4 Extending the COI Using Abstraction . . . . . 8
2.3 Datapath and Control Logic Interface . . . . . 9

3 Adding an Efficient Set of Assertions . . . . . 11

3.1 Relevant Research on Adding and Optimizing Assertions . . . . . 11
3.2 Methods Investigated to Add Assertions . . . . . 13

3.3 1-1 Mapping . . . 13
3.3.1 Applying 1-1 Mapping to a Specification Document . . . 14
3.3.2 Requirements, Advantages and Drawbacks of 1-1 Mapping 19
3.4 Divide and Conquer: Extended List of Assertion Sub-Categories . 21
3.4.1 Behavioral Assertions . . . 21
3.4.2 Invariant Assertions . . . 22
3.4.3 Temporal Assertions . . . 22
3.4.4 Abstract Assertions . . . 24
3.4.5 Complex Assertions and Combinations of Assertions . . . 25
3.4.6 Other Types of Assertions . . . 26

4 The State Table and Boolean Functions . . . 27

4.1 The State Table Method . . . 27
4.1.1 ST Drawbacks . . . 29
4.2 The Boolean Function Method . . . 30
4.3 Advantages and Features of the ST and Boolean Function . . . 31

5 Writing Sufficient Assertions in Verification . . . 35

5.1 When Have We Written Sufficient Assertions? . . . 35
5.1.1 Assertion Density and Fault Coverage . . . 35
5.1.2 Writing Sufficient Assertions on a Type by Type Basis . . . 35
5.2 Who Should Write Assertions? . . . 40

6 Optimizing Assertions using Logic Synthesis Techniques . . . 43

6.1 Minimizing Assertions by Optimizing the Boolean Function . . . 44
6.2 Combining Multiple Assertions . . . 45
6.2.1 Combining Assertions using Common Cubes . . . 45
6.2.2 Merging Multiple Karnaugh Maps . . . 47
6.2.3 Heuristic Techniques . . . 53
6.2.4 Combining Assertions using Temporal Operators . . . 60
6.2.5 State Variables as Karnaugh Map Outputs . . . 61
6.3 Overview of the Various Methods to Used Combine Assertions . . 64
6.3.1 Assertions Obtained from the Various Methods . . . 64
6.3.2 Comparison of the Various Methods . . . 64
6.4 Generating Invariant Assertions . . . 69
6.4.1 Obtaining Invariant Assertions from the Merged Karnaugh Map . . . 69

6.4.2 Determining Unreachable States . . . . . 72

7 IFV Results and Time Resources . . . . . 75

7.1 Varying the length of the data structure . . . . . 75
7.2 Handful of assertions could consume the vast majority of resources 77
7.3 CPU Time for Each Method . . . . . 77

8 Conclusions and Future Work . . . . . 86

8.1 Summary . . . . . 86
8.2 Future Work . . . . . 87

Appendix A: Heuristics Applied to FEOU . . . . . 89

Appendix B: FEOU Assertions Obtained from the Various Methods . . . . . 90

References . . . . . 93

LIST OF TABLES


| Table | | page |
|---|---|---|
| 3–1 | A Simplified FIFO Controller Specification Document | 15 |
| 3–1 | A Simplified FIFO Controller Specification Document | 16 |
| 3–2 | Control Statements of the FIFO Controller Specification Document | 17 |
| 3–3 | Assertions Describing the Control Statements of the FIFO Controller Specification Document | 18 |
| 3–4 | Requirements, Advantages and Drawbacks of 1-1 Mapping | 19 |
| 3–4 | Requirements, Advantages and Drawbacks of 1-1 Mapping | 20 |
| 3–5 | Assertions for *FULL* | 22 |
| 3–6 | Examples of Invariant Assertions | 22 |
| 3–7 | Examples of Bounded Temporal Assertions | 23 |
| 3–8 | Examples of Unounded Temporal Assertions | 24 |
| 3–9 | Examples of Temporal-Invariant Assertions | 25 |
| 4–1 | Synchronous FIFO State Table for signal *full* | 28 |
| 4–2 | ST Derived assertions for *FULL* | 28 |
| 4–3 | FIFO ST for signal *FULL* with best ordering | 29 |
| 4–4 | FIFO ST for signal *FULL* without proper ordering | 29 |
| 4–5 | State Table for signal *StateFSM* | 32 |
| 4–6 | ST for *OUT = (XOR(A,B,C) AND D)* | 33 |
| 4–7 | ST for *OUT = (XOR(A,B,C) AND D)* using a sub-function | 33 |

4–8 Single ST for *grant_A*of a round-robin arbiter . . . . . 34
4–9 Merged ST for *Grant[0:N-1]* of a round-robin arbiter . . . . . 34
5–1 Examples of Project-Specific Invariant Assertions . . . . . 37
5–2 Simplifying Complex Assertions . . . . . 39
5–3 Who Should Write Assertions Summary . . . . . 41
6–1 g and h Assertions . . . . . 46
6–2 List of Assertions Obtained When Merging the g and h Karnaugh Maps 48
6–3 Mathematical Equivalence of Merging Karnaugh Maps . . . . . 48
6–4 FIFO Functions and Inverse Functions . . . . . 50
6–5 Minimized Functions for A, B and C . . . . . 54
6–7 ABC Assertion Table Comparison 1 . . . . . 58
6–8 ABC Assertion Table Comparison 2 . . . . . 59
6–9 Using Temporal Operators to Combine Assertions . . . . . 60
6–10 Using state variables as Karnaugh map outputs to minimize assertions 62
6–11 Total Number of Assertions Obtained by Each Method . . . . . 65
6–12 Comparison of the Various Optimizations . . . . . 66
6–12 Comparison of the Various Optimizations . . . . . 67
6–12 Comparison of the Various Optimizations . . . . . 68
6–12 Comparison of the Various Optimizations . . . . . 69
7–1 Increasing the FIFO Size with effort=30min and a total of 51 assertions 76
7–2 Increasing the effort with FIFO Size set to 32 with a total of 51 assertions 77
7–3 Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 4 . . . . . 77

7–3 Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 4 . . . . . 78

7–4 Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 8 . . . . . 78

7–5 Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 16 . . . . . 78

7–5 Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 16 . . . . . 79

7–6 CPU Time per Assertions for FIFO_Size=16 with minimized functions 80

7–7 CPU Time per Assertions for FIFO_Size=16 with heuristics . . . . . 81

7–8 CPU Time per Assertions for FIFO_Size=16 with Merged Karnaugh Maps . . . . . 81

7–8 CPU Time per Assertions for FIFO_Size=16 with Merged Karnaugh Maps . . . . . 82

7–9 CPU Time per Assertions for FIFO_Size=16 with Past State Variable Minimized Functions . . . . . 83

7–10 CPU Time per Assertions for FIFO_Size=16 with Past State Variable Heuristics . . . . . 84

7–11 CPU Time per Assertions for FIFO_Size=16 with Past State Variable Merged Karnaugh Maps . . . . . 84

8–2 FEOU Assertions Table 1/5 . . . . . 90

8–3 FEOU Assertions Table 2/5 . . . . . 91

8–4 FEOU Assertions Table 3/5 . . . . . 91

8–5 FEOU Assertions Table 4/5 . . . . . 92

8–6 FEOU Assertions Table 5/5 . . . . . 92

LIST OF FIGURES


Figure page

2–1 Extending assertions spatially for the arbiter block . . . . . 7

6–1 Karnaugh map for the Full signal . . . . . 44

6–2 Karnaugh Maps for the g and h functions . . . . . 46

6–3 Merged Karnaugh map for 'g' and 'h' . . . . . 48

6–4 Karnaugh map for Full . . . . . 49

6–5 Karnaugh map for Empty . . . . . 49

6–6 Karnaugh maps for Overflow and Underflow . . . . . 50

6–7 Merged Karnaugh map for Full and Overflow . . . . . 51

6–8 Merged Karnaugh map for Full and Empty . . . . . 52

6–9 Merged Karnaugh Map for Full and Underflow . . . . . 52

6–10 Merged Karnaugh Map for Full, Empty, Overflow and Underflow . . . 53

6–11 Karnaugh Maps for A, B and C as Well as the Merged Karnaugh Map 55

6–12 Identifying Common Cubes in Karnaugh Maps . . . . . 56

6–13 Identifying Cube Containment in Karnaugh Maps . . . . . 56

6–14 Applying Multiple Reductions on Cubes . . . . . 57

6–15 Karnaugh maps using dependent outputs for F,E,O and U . . . . . 61

6–16 Merged Karnaugh Maps Using Dependent Outputs for F,E,O and U . 63

6–17 Merged Karnaugh Maps Using Dependent Outputs for FE, FO and FU 63

6–18 Merged Karnaugh Maps Using Dependent Outputs and Heuristics . . 64

6–19 Prime Invariant Assertions using Quine-McCluskey . . . . . 71
6–20 State Diagram with Unreachable States . . . . . 73
6–21 Merged Karnaugh Map After Removing Unreachable States for FEOU 74
8–1 Heuristics applied to FEOU . . . . . 89

# CHAPTER 1
# Introduction

## 1.1 Hardware verification

Verification of hardware electronics is a major task in producing a quality product set out by the architects and designers and it is essential to get the entire system working as intended prior to production and sale. Verification is essential to guarantee that a product meets it's requirements and fulfills its intended purpose. [54]

In the recent past electronic designs have become more complex. This complexity has translated itself into even more complex verification components and environments. Verification has become a very critical segment of complex system and electronic chip designs. The challenge of verification is to raise the quality of the verification process while at the same time reducing the time it takes to validate the design before fabrication. [53]

The main techniques of hardware verification are functional verification and formal verification.

Functional verification procedures involve performing special tests to model or simulate the design. Functional verification procedures usually involve repeating random and constraint tests an extensive number of times, reviewing the simulation model, debugging the test waveforms of the complex hardware and software, and coverage checking...

Formal verification on the other hand is the act of proving or disproving the correctness of logical algorithms underlying a system with respect to a certain formal specification or property, using pure mathematical techniques. [3]

Assertions (the main theme of this thesis) are exceptionally useful as they can be used in both functional and formal verification.

## 1.2 Research Collaboration with PMC-Sierra

Between July 2010 and January 2011, PMC-Sierra and the Integrated Microsystems Laboratory (IML) at McGill University worked in collaboration on a research project entitled "Semi-Formal Verification using Assertion-based Techniques". The goal of the project was to investigate the application of the assertion-based verification (ABV) in industrial circuit designs. The project addressed the following questions:

- When have enough assertions been written?
- How can it be determined whether additional assertions can be added to an existing set without adding any redundancies?
- How can ABV be used to complement other forms of verification?
- How should assertions be written and by who? Designers or Verifiers?
- How can one measure the coverage and effectiveness of the written assertions?

To help answer these questions, assertions were written and verified using IFV (the Cadence tool for formal verification) for various RTL designs, some of which had previously been verified by PMC-Sierra, in an attempt to gain insights into answering the questions above. The project was implemented on the following circuit blocks:

1. Synchronous FIFO Controller
2. Asynchronous FIFO Controllers (Gray Code FIFO and Ping-Pong FIFO)
3. MS-Bus to AMBA-AHB Bridge Telecom System Block
4. AHB-Master interface

## 1.3 Purpose and Organization of the Thesis

The purpose of this thesis is to provide new ideas on how to resolve some of the major issues involving assertion based verification namely providing a non-redundant set of assertions and increasing the overall speed and efficiency of the assertion set. To do so, different methods of adding assertions to an existing set are presented and the task of writing enough assertions are discussed.

The thesis is organized in the following manner: chapter 2 provides a background on ABV, chapter 3 Boolean various methods to add non-redundant assertions, chapter 4 presents the state table and Boolean function methods of adding assertions, chapter 5 tackles the question "when have we written sufficient assertions?", chapter 6 presents new methods to optimize assertions by detecting and taking advantage of common features in assertions, chapter 6 also shows how invariant assertions can be automatically generated, chapter 7 shows the IFV results including the CPU time for different sets of assertions and chapter 8 wraps up the thesis with a conclusion and future work.

# CHAPTER 2
# Background on Assertion Based Verification

## 2.1 Background on Assertions

### 2.1.1 What is an Assertion?

An assertion is a statement about a design's intended behavior which must be verified, it describes what behavior is expected when given input is presented to the design [28] [2].

There are multiple languages available for writing assertions such as the Open Verification Library (OVL), SystemVerilog Assertions (SVA) and Property Specification Language (PSL). This thesis will focus primarily on PSL assertions written in Verilog. The assertion language selected does not affect in any way the results obtained and presented in this thesis.

An example of a PSL assertion is:

```
a_overflow : assert always {fifo_full |-> !wr_fifo}
                    abort reset @(posedge clk);
```
[51]

An assertion usually contains a label to indicate the name given to the assertion (a_overflow), an occurrence operator (always, never, eventually, once...), an enabling condition (fifo_full in the above example), an implication operator (logical if, next[n], $|->$ or $|=>$), a fulfilling condition (!wr_fifo in the above example), a discharging

condition (until, abort) and the clock expression used to sample the assertion [18], [19].

### 2.1.2 Evolution and Maturity of ABV

The rapid growth in complexity of hardware designs has given a need for a better verification methodology, one with improved observability within the design and controllability of the verification process [20].

Up to 80% of the overall design costs are due to verification [24] and assertion-Based Verification (ABV) has been identified as a modern, powerful verification concept that can ensure enhanced productivity, higher design quality and faster time to market. [20]

ABV has gained a very strong momentum over the last decade, as evident by the increasing number of verification reports, books, commercial offerings and academic research efforts in this area. Many prominent companies use assertions with great success as part of their verification methodology including Cisco Systems, Hewlett-Packard, IBM, Intel, Motorola, Silicon Graphics... [28].

### 2.1.3 Advantages of Assertions

The major advantage of using Assertion Based verification (ABV) is the reduction in time to market of a product. Assertions detect design errors as soon as they trigger without waiting for the effects to propagate to the outputs of the device. This can save hours of trying to look backwards to get to the source of the problem [18] [2]. The use of assertions can save up to 50 percent of debug time [28].

Other advantage of assertions include convenience, re-usability, flexibility and simplicity. Assertions can be added at any stage of the design and verification process and in any quantity. They can be added incrementally as needed or time permits. Assertions can be reused across different stages of the design as well as in different projects. They can be checked in either dynamic simulations or formal verification and even a single assertion can help find design errors [11] [28] [2].

### 2.1.4 Types of Assertions Written in Verification

Many sources in the assertion based verification world identify two main types of assertions: immediate assertions and concurrent assertions [51], [14], [52], [19], [36]. Concurrent assertions describe design behaviors over time while immediate assertions are not temporal in nature and are evaluated immediately [52], [14].

Although temporal properties share many similarities with assertions, two main types of properties stood out in ABV sources: safety and liveness properties [18], [14], [40]. Safety properties check that something bad never happens whereas liveness properties check that something good eventually happens [14].

## 2.2 Extending Assertions

Assertions can be extended spatially, temporally and combinatorially. Furthermore, abstraction can be used to increase the cone of influence (COI) of an assertion.

### 2.2.1 Extending Assertions Spatially

As a simple example of extending assertions spatially, assume that a set of assertions for the arbitration of request and grant have been written inside of the block labeled "Arbiter" in Figure 2–1. Extending the assertions spatially would be to write the same set of assertion but by taking the request and grant signals from within their respective blocks (BlockA, BlockB...).

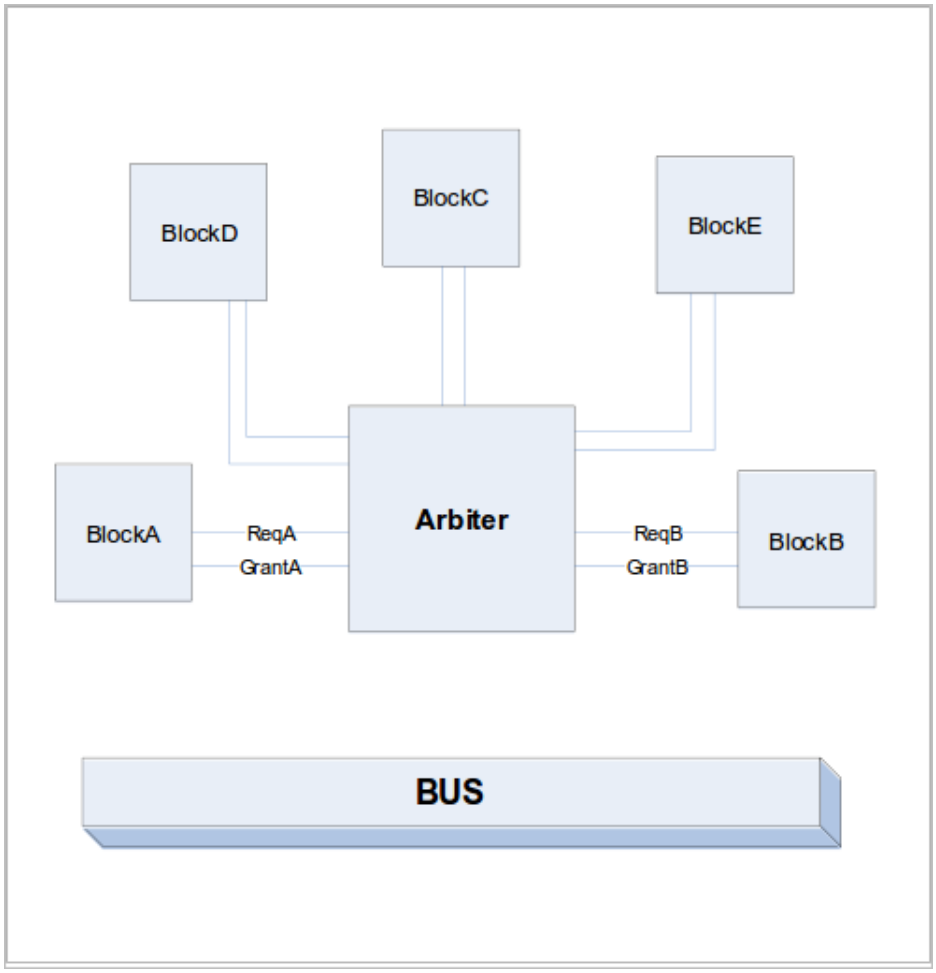



Figure 2–1: Extending assertions spatially for the arbiter block

Extending the assertions spatially can verify that the blocks are interfaced correctly while verifying that simple errors were avoided.

### 2.2.2 Extending Assertions Temporally

Signals that have time delays can be extended temporally as is the case for a `full` signal that is delayed by one clock cycle inside the write interface block before it reaches the FIFO.

Extending assertions temporally would come down to adding a [*delay] to the assertion. As with spatial assertions, extending assertions temporally serves the purpose of avoiding errors.

### 2.2.3 Extending Assertions Combinatorially

Outside of the Synchronous FIFO block, the *enqueue* signal is assigned to *enqueue = (write_request AND !full)*. Extending assertions combinatorially would be to replace *enqueue* in the *full* assertion above with its combinational equivalent *(write_request AND !full)*.

### 2.2.4 Extending the COI Using Abstraction

Assertions can be written higher up in hierarchy to cover a larger Cone of Influence (COI). The purpose of extending the COI up the hierarchy levels is to increase the amount of verification coverage of each individual assertion. Increasing the COI can extend the assertion coverage across many components and subcomponents.

In formal verification, the size of the COI of an assertion can have adverse affects on the computational effort needed to prove if the assertion is true or false. A formal verification attempts to mathematically prove whether a statement is true or false over an entire simulation space for all possible input combinations. When the COI of an assertion is increased then the formal verification tool must run over a larger simulation space.

Increasing the COI in a dynamic verification setting does not suffer the scalability issues that exist in formal verification because in dynamic verification the simulation is constantly running and only one input combination is being tested at a given time.

In many cases formal verification and simulation impose different requirements on assertions for efficiency considerations. By following tool-specific recommendations about assertion efficiency, possibilities exist to make a small sacrifice in formal verification efficiency that can provide a tremendous boost in dynamic simulation speed [14].

### 2.3 Datapath and Control Logic Interface

Assertions are known to be well suited to verify the control logic but not well suited to verify the datapath logic. Examples of designs that can benefit from the use of assertions include controllers (memories, FIFOs, DMA's, bus interfaces) finite state machines, black box verification, system-level requirements (latency, processing algorithms and mission critical requirements) [18] [7].

However, the datapath and the control logic are not always completely separate from each other and in certain cases, the datapath and the control logic can affect each other. The optimal solution at the interface between datapath and control logic may vary depending on the verification environment (formal vs dynamic).

In a formal setting, the number of states required to verify a datapath heavy block increases exponentially at the interface between control logic and datapath. The datapath will have a tendency to dominate over the control logic. However, verifiers can sometimes limit the effect of the datapath in mixed control blocks by temporally assigning the data width to a low value in formal settings. The control logic may then be verified without consuming too much resources.

# CHAPTER 3
# Adding an Efficient Set of Assertions

## 3.1 Relevant Research on Adding and Optimizing Assertions

One of challenges of ABV is that enabling assertions generally requires 10-30% more computing time per simulation run [28]. Many authors have recognized the need to optimize the assertion set available and to provide a more efficient set of assertions. They recommend adopting an ABV methodology as a necessary means of adding assertions [28] [19] [18] and that ABV itself is regarded as a verification and a documentation methodology [19].

The ABV verification methodology includes the following recommendations:

1. The existence of design documents and specifications for identifying where to add assertions. Buy-in from all members of the project team is essential when attempting to implement assertion methodologies.
2. Assertions can and should be added before the verification step and should be part of the design process [11]. Writing assertions is a continuous process and assertions must be written along with the RTL code. Assertions can even be written in the place of RTL comments.
3. Failures that are not identified by assertions must be analyzed to determine whether assertions can be written to detect the problems.

Other works such as [31] and [35] attempt to automate the generation of assertions. The paper in [31] presents an optimization algorithm which attempts to automatically define the most efficient set of assertions and determine the location of these assertions to be inserted in the design. The algorithm is based on intelligent randomness (the new assertions are generated randomly), gradual progress (the methods improve gradually), and fitness (the methods utilize a fitness function to compare the efficiency of the results). However, in order to calculate assertion coverage, all possible faults are injected into the faulty circuit [31].

The authors of [43] developed an algorithm for writing non-overlapping assertions. The algorithm starts by finding the coverage of each assertion on the design errors and distinguishes between essential and futile assertions. A greedy algorithm is then used to find the smallest set of assertions that covers as many design errors as possible.

The CheckSpec tool in [4] verifies whether a given assertion suite is free from inconsistencies and quantifies the completeness of a given assertion suite.

A framework for debugging and correcting PSL assertions is proposed in [29]. The failing assertions, the corresponding counter-examples along with a mutation model are used to produce closely related assertions that are verified against the design. These properties are generated by an input set of modifications that mutate existing assertions and provide both suggestions for possible corrections as well as insights into the design behavior.

Normally when an assertion violation occurs, it is standard to terminate the program, debug it if possible, and re-execute. Instead, [25] propose the use of the violated assertion as a basis for repairing the state of the program and let it continue.

According to [19], structural properties can be automatically extracted using tools and are different from functional properties. Structural properties include information about the structure of the RTL such as: deadlock states where 2 FSM are waiting for each other, terminal states where a state cannot be exited, unreachable states where a state that cannot be entered, clock crossing domain instability issues, and linting (language guideline violation).

## 3.2 Methods Investigated to Add Assertions

Several promising methods to add assertions to an existing set of assertions were investigated including 1-1 mapping and a divide and conquer approach complemented with the use of state tables (ST) and the manipulation of Boolean functions. These methods are outlined, compared to each other and a discussion of their drawbacks and advantages is presented.

## 3.3 1-1 Mapping

The 1-1 mapping method was investigated as a means of adding assertions to an existing set of assertions without adding redundancies. The premise of the mapping process is that every statement in the specification document that can be described by an assertion should be covered by one assertion and every written assertion should be matched to a statement in the specification document.

A typical specification document contains a variety of information regarding a component. It is common to find, within the specification document an introduction, a list of the main features, a list of I/O ports, a description and examples of how the component is intended to function, state machines, waveforms...

### 3.3.1 Applying 1-1 Mapping to a Specification Document

The process of 1-1 mapping begins by identifying the control logic information available within the specification document and mapping the control logic information to the assertions files.

Consider an example of a simplified FIFO controller specification document in Table 3–1.

# A Simplified FIFO Controller Specification Document

Cover Page

Synchronous FIFO Controller v024
Issue 1.3 May 2005
(...)

Table of Contents (...)

Ch 1) Introduction

This document provides information concerning the Synchronous FIFO Controller and is divided into the following subsections:
(...)

Ch 2) FIFO Controller Features (...)

Ch 3) I/O Ports (...)

Ch 4) Example Application (...)

Ch 5) (...)

Ch 6) FIFO Controller Logic
(Finally we reach a section with information regarding the control logic)

This chapter describes the behavior of the FIFO controller.
(...)

6.1) FIFO write
The wr_addr signal increments at the next rising edge of the clock when both enqueue is true and full is false.

Table 3–1: A Simplified FIFO Controller Specification Document

# A Simplified FIFO Controller Specification Document

In the example shown in Figure (...) (irrelevant information with respect to writing assertions not shown for simplification)

6.2) FIFO full

The full signal is asserted when wr_addr and rd_addr point to the same address.

The full signal will remain asserted until the next rising clock edge after dequeue is high. When full is asserted, the next write attempt without a read in between will be unsuccessful.

6.3) FIFO overflow

The overflow signal is asserted when both full is true and enqueue is true
(...)

6.4) FIFO read (...)
6.5) FIFO empty (...)
6.6) FIFO underflow (...)
6.7) Packet error (...)
6.8) (...)
(...)

Ch 7) (...)

Ch 8) May describe additional control logic (...)

Ch 9) (...)

Ch 10) Additional Information
(...)

Table 3–1: A Simplified FIFO Controller Specification Document

If an assertion file for the FIFO Controller did not already exist then 1-1 mapping would involve identifying the control statements that can be described as assertions. As an example, from the specification document of Table 3–1, the statements that can be covered by assertions are shown in Table 3–2 and the resulting assertions for these statements are shown in Table 3–3.

| |
|---|
| 6.1) FIFO write<br>assert_wr_ad_p1: The wr_addr signal increments at the next rising edge of the clock when both enqueue is true and full is false.<br><br>In the example shown in Figure (...)<br><br>6.2) FIFO full<br>assert_full_r: The full signal is asserted when wr_addr and rd_addr point to the same address.<br><br>assert_full_f, assert_full_h: The full signal will remain asserted until the next rising clock edge after dequeue is high. assert_wr_ad_h: When full is asserted, the next write attempt without a read in between will be unsuccessful.<br><br>6.3) FIFO overflow<br>assert_overflow_r: The overflow signal is asserted when both full is true and enqueue is true |

Table 3–2: Control Statements of the FIFO Controller Specification Document

```
assert_wr_ad_p1: assert always {enqueue = '1' AND
         full = '0'} |=> { wr_addr = wr_addr + 1 };

assert_full_r: assert always {rd_addr = wr_addr + 1 AND
         enqueue = '1' AND dequeue  = '0'}
         |=> {full = '1'};

assert_full_h: assert always {full = '1' AND dequeue  = '0'}
         |=> {full = '1'};

assert_full_f: assert always {full = '1' AND dequeue  = '1'}
         |=> {full = '0'};

assert_wr_addr_s: assert always {full = '1' AND enqueue  = '1'}
         |=> {wr_addr = prev(wr_addr)};

assert_overflow_r: assert always {overflow = '0' AND
         full = '1' AND enqueue = '1'} |=> {overflow = '1'};
```

Table 3–3: Assertions Describing the Control Statements of the FIFO Controller Specification Document

As mentioned above, 1-1 mapping is applied when an assertion file does not already exist... If on the other hand, an assertion file does already exist (and 1-1 mapping had not been used) then 1-1 mapping would be useful to link the assertions in the assertion file back to the specification document. Thus, additional assertions could then be written for the remaining control statements found in the specification document that were not covered by assertions.

To explain the meaning of the assertion language used... assert_wr_ad_p1 is the name of the assertion, "assert always" means check the statement at every clock

cycle (or whatever the moment of time is defined as), {enqueue = '1' AND full = '0'} is the boolean condition, `|=>` means that at the next clock cycle the following condition {wr_addr = wr_addr + 1} will be true. Note that `|->` is similar to `|=>` except that with `|->` the following condition occurs in the same clock cycle as the previous condition.

### 3.3.2 Requirements, Advantages and Drawbacks of 1-1 Mapping

The following requirements, advantages and drawbacks in Table 3–4 were observed while investigating and applying 1-1 mapping on several control blocks including the synchronous FIFO controller.

| **Requirements** | **Advantages** | **Drawbacks** |
|---|---|---|
| A Specification document (spec) must exist for 1-1 mapping to be applied | A spec helps organize and explain the intentions of a block | Not every block has a spec and writing one for each block is time consuming |
| The Specification document must be detailed and at the very least contain all relevant control logic | Mistakes stand out more clearly when reviewing the spec vs the assertion file<br><br>Redundancies are easier to spot in plain English | The process of ensuring the spec is sufficiently detailed is time consuming |

Table 3–4: Requirements, Advantages and Drawbacks of 1-1 Mapping

| Requirements | Advantages | Drawbacks |
|---|---|---|
| Complex assertions must be simplified if possible | Simple assertions are easier to debug and review | Not all complex assertions can be simplified which raises the issue of how to describe them in the spec |
| The spec must be properly maintained and updated<br><br>All control logic in the spec must be identified and if additional assertions are written then statements must be added to the spec | Helps identify additional assertions that may have been missed if 1-1 mapping was not applied | If assertions are modified then the spec has to be updated<br><br>Must simultaneously update 2 different documents (the spec and the assertion file) |

Table 3–4: Requirements, Advantages and Drawbacks of 1-1 Mapping

As a summary, 1-1 mapping is a useful technique to add and organize assertions. However, there are significant drawbacks to the 1-1 mapping method which limits its usability in a practical verification environment. There is no guarantee that every RTL block will have its own specification document and maintaining all these specification documents for every RTL block is tedious and time consuming.

## 3.4 Divide and Conquer: Extended List of Assertion Sub-Categories

The drawbacks inherent to 1-1 mapping led to a search for alternative methods to add assertions to an already existing set of assertions. While researching how to optimize ABV, the need arose to identify the different types of assertions written in verification. The line of thought was that if the various types of assertions can be identified, then a divide and conquer approach could be applied to addressing the question "when have sufficient assertions been written?".

Even though assertions can be labeled as either immediate or concurrent as described in section 2.1.4, such a list becomes insufficient when describing the various roles of assertions in a practical setting. The goal of the following sections is to present a more practical list of assertions from the points of view of the people writing the assertions such as the designers and verification engineers. Several sources such as [30], [46], [23] and [55] have been helpful in producing such a list.

It is worth noting that the different types of assertions listed are not intended to be strictly defined. The types of assertions are mainly intended to be used by verification engineers in order to take full advantage of the benefits of assertions.

### 3.4.1 Behavioral Assertions

Simply put, a behavioral assertion is an assertion that describes the events that will directly cause a Boolean function to be eihter at logic-1 or logic-0.

Assuming the Boolean function for Full is:

`full = !read AND write AND {write_addr+1=read_addr} + !read AND full`,

examples of behavioral assertions for full can be found in table 3–5.

| assert always {Full==0 AND Write==0} \|⇒ {Full==0}; |
|---|
| assert always {Read==0 AND Full==1} \|⇒ {Full==1}; |
| assert always {Read==1} \|⇒ {Full==0}; |

Table 3–5: Assertions for *FULL*

### 3.4.2 Invariant Assertions

An invariant assertion is a statement that describes a time-independent relationship involving multiple signals. Examples of invariant assertions can be found in table 3–6.

| assert never {full==1 AND empty==1}; |
|---|
| assert always {empty==1 XNOR (read_addr==write_addr)}; |
| assert never {write_addr > end_addr}; |
| assert always {!a AND b AND c}; |

Table 3–6: Examples of Invariant Assertions

Further subcategories of invariant assertions include: (a) Invariants from the interaction of RTL functions such as the relationship between full and empty, (b) project specific requirements such as the requirement that a read and a write attempt cannot occur simultaneously and (c) environment specific invariants such as the case when a burst signal will never be enabled in the optical block in the OTN subsystem but will be enabled in the DFT subsystem.

### 3.4.3 Temporal Assertions

Temporal assertion can be divided into 2 major groups: temporally bounded assertions and unbounded assertions. As is somewhat evident, these assertions are

distinguished based on whether they are bounded by a limited number of clock cycles or an infinite number of cycles.

Many temporally bounded assertions have well defined practical applications as shown in table 3–7.

| | |
|---|---|
| **Ensure fairness** | `{assert always {!reqA, reqA} \|-> {!grantA[*0:24]; grantA}}` |
| **Verify an expected event** | `{assert always {state = init} \|=> { (state/=end)[*4:7]; state=end};}` |
| **Ensure the timely arrival of a signal** | `assert always {write_interface_en = '1'} \|=> {[*1:4]write_fifo};` |
| **Ensure that a Protocol is met** | `rose(write) \|=> stable(addr)[*3];` |

Table 3–7: Examples of Bounded Temporal Assertions

To explain the meaning of some of the assertion jargon... the comma in `{!reqA, reqA}` means at the next clock cycle, so the statement means that if reqA is low and the next cycle reqA is high then check the following condition. `!grantA[*0:24]; grantA` means grant is low between 0 and 24 cycles but after which grantA must be high. stable(addr)[*3] means that addr is stable for exactly 3 clock cycles. "rose" means rising edge.

Unbounded assertions are generally used whenever the specified timing-window is too large. Large bounds can cause model checking tools (such as the Cadence formal verification tool, IFV) to consume longer computation time. According the IFV documentation from Cadence, an unbounded version of the assertion is simpler to verify in formal verification tools (such as IFV) compared to their bounded counterpart if the bound is too large ($> 8$ cycles). Some examples of unbounded temporal assertions can be found in table 3–8.

| | |
|---|---|
| **Detect starvation** | `{reqA} \|-> {eventually! gntA}` |
| **Detect deadlock** | `{state==init} \|-> {eventually! state==end};` |

Table 3–8: Examples of Unounded Temporal Assertions

### 3.4.4 Abstract Assertions

An abstract assertion is an assertion that hides a deeper understanding of the RTL by abstracting the details of the underlying RTL. The complexity of abstract assertions can range from simple to very complex. An example of a simple abstract assertion is as follows:

```
assert always ((dequeue = '0' AND overflow = '1' AND enqueue = '1')
    |=> (full_int = '1') abort (fifo_clrb='0' OR packet_error='1');
```

The abort statements means that if the following condition occurs, then the assertion will be ignored.

Although the assertion described above looks like a behavioral assertion, it is not considered as such because `overflow` does not cause `full` to be set to 1. In more general terms, one of the signals essential to triggering the assertion (overflow is considered essential here because removing overflow will give an untrue assertion) is not a cause in setting the function to '1', rather overflow and full are linked in a more "abstract" manner.

To understand how `full` and `overflow` are linked consider the following: when the FIFO is in `overflow`, then either the FIFO is still `full` (in the case of a write without a read to the FIFO when the FIFO was already full) or the FIFO was full at the previous clock cycle but a simultaneous read and write attempt deset the full

signal while setting overflow. With an enqueue and a lack of dequeue both scenarios will lead to `full` being set.

### 3.4.5 Complex Assertions and Combinations of Assertions

A complex assertion is an assertion that combines several types of assertions. A simple example of a complex assertion is to merge multiple behavioral assertions into a single assertion by using the "OR" expression.

```
assert always ({( (dequeue = 0 AND full = 1) OR
         (dequeue = 0 AND enqueue = 1 AND overflow = 1) OR
         (dequeue = 0 AND enqueue = 1 AND (read addr - write addr = 1)) )}
 |=> {full = 1 }) abort (fifo clrb=0 OR packet_error=1);
```

Another example of a complex assertion is a flow assertion. A flow assertion is produced when one event causes a whole series of events to occur. For example, when a memory write instruction is received, then the following series of events occurs.

```
assert always {instr = write} |=> {mem_cs ; mem_wr ; !mem_wr};
```

Other complex assertions may be produced by combining various types of assertions. An example of that is an invariant assertion that is only observed within a specific time period. Examples are found in table 3–9.

| |
|---|
| {start} \|-> {(!finish AND cond)[*]; finish AND cond} |
| {a && !c} \|-> {(!c && b_is_zoh)[*]; c && !b_is_zoh};} |

Table 3–9: Examples of Temporal-Invariant Assertions

### 3.4.6 Other Types of Assertions

Although an effort was made to present a complete overview, there might still be additional types and subtypes of assertions used in a practical verification environment that were not mentioned.

# CHAPTER 4
# The State Table and Boolean Functions

The original motivation behind the divide and conquer methodology was the discovery that state tables can be used to add efficient and non-redundant assertions, where the assertions obtained by using state tables were of a very specific type of assertion labeled as behavioral assertions.

## 4.1 The State Table Method

An example of using the ST (State Table) method to add behavioral assertions is shown in Table 4–1. Table 4–1 represents the state table of the control signal `full` of a synchronous FIFO controller. Signals that are dependent on `full` are listed on the left side of the double vertical line in the table while the next value that `full` will take at the next clock cycle is on the right side of the double line. There are three signals and one expression that describe the behavior of `full`, namely `dequeue`, `full`, `enqueue` and `write_addr + 1 = read_addr`. Each row in the state table can be regarded as a separate and distinct assertion.

The *don't cares* ($\times$) allow a particular dependent signal to have any logic value. The ST method has the added benefit of minimizing the amount of redundant literals that are contained inside of the Boolean expressions (conditions) of the assertions.

| **Dependent Signals** | | | | **Control Signal** |
|---|---|---|---|---|
| **dequeue** | **full** | **enqueue** | **write_addr + 1 = read_addr** | **full** |
| 0 | 0 | 0 | × | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | × | × | 1 |
| 1 | × | × | × | 0 |

Table 4–1: Synchronous FIFO State Table for signal *full*

To achieve a compact set of assertions, only the dependencies that drive `full` are considered. Each row in the ST describes the conditions required for the full signal to be either in a high or low state. For this case, only 5 assertions are needed to fully cover the full signal, and are listed in Table 4–2. These assertions effectively cover all the required conditions to assert either a logic-0 or a logic-1.

| **Compact Assertions for Full** |
|---|
| {dequeue == 0 && full == 0 && enqueue == 0} \|⇒ {full == 0} |
| {dequeue == 0 && full == 0 && enqueue == 0 && (write_addr + 1 = read_addr) == 0} \|⇒ {full == 0} |
| {dequeue == 0 && full == 0 && enqueue == 1 && (write_addr + 1 = read_addr) == 1} \|⇒ {full == 1} |
| {dequeue == 0 && full == 1} \|⇒ {full == 1} |
| {dequeue == 1} \|⇒ {full == 0} |

Table 4–2: ST Derived assertions for *FULL*

### 4.1.1 ST Drawbacks

The ST method has a few drawbacks worth mentioning. The length of the ST (and therefor the number of assertions) is heavily affected by the ordering of the dependent signals.

To visualize this, Tables 4–3 and 4–4 show examples of equivalent state tables where the only difference is in the ordering of the columns.

| **dequeue** | **full** | **enqueue** | **write_addr + 1 = read_addr** | **full** |
|---|---|---|---|---|
| 0 | 0 | 0 | × | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | × | × | 1 |
| 1 | × | × | × | 0 |

Table 4–3: FIFO ST for signal *FULL* with best ordering

| **enqueue** | **full** | **write_addr + 1 = read_addr** | **dequeue** | **full** |
|---|---|---|---|---|
| 0 | 0 | × | × | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | × | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 |

Table 4–4: FIFO ST for signal *FULL* without proper ordering

It is worth noting that the state table in Table 4–3 is a lot more compact than the state table in Table 4–4. If each row were to depict an assertion as we mentioned earlier, then the non-optimized ordering will lead to an increase in the number of assertions from 5 to 12 assertions compared with the optimized ordering state table. Column ordering must be implemented to efficiently use the ST.

Another major limitations with using the ST method is that as we continuously insert a dependent signal, the number of rows, in the unreduced state table (or it's truth table), will double. A truth table with a large number of dependent signals (e.g., 10 dependencies) can create a large number of entries (e.g., $2^{10}$ entries in the worst case scenario).

To reduce the size of the state table, proper column organization and the use of don't cares is a must. In certain cases, it may be found that the ST cannot be simplified or significantly reduced using don't cares. In such cases it is infeasible to apply the ST method.

## 4.2 The Boolean Function Method

An alternative method to manipulating state tables would be to work directly with the Boolean function of the control signals. Similar to the state table method, the Boolean function and inverse function of a signal can be used to obtain a complete set of behavioral assertions with respect to the signal involved. The Boolean function can help both designers and verifiers to write compact assertions as they ensure that the assertions will include only the necessary signals of the appropriate control logic. An example is shown below for signal `full` from the FIFO synchronous controller:

```
full = !dequeue.enqueue.{write_addr+1=read_addr} + !dequeue.full
!full = dequeue + !{write_addr+1=read_addr}.!full + !enqueue.!full
```

To simplify the notation and improve readability, these functions can be written as:

```
F = !R.W.A + !R.F
!F = R + !A.!F + !W.!F
```

In the above functions, each implicant can be seen as a separate assertion. Both the Boolean functions and its inverse are needed to obtain a complete set of behavioral assertions with respect to the signals that they are applied to.

### 4.3 Advantages and Features of the ST and Boolean Function

In both the ST and Boolean function, dependent signals can be binary, ternary, quaternary or N-valued logic. The `StateFSM` signal in Table 4–5 is an example of a ST with N-valued logic as it can take the values ranging from S0 through S4.

| Dependent Signals | | | Control Signal |
|---|---|---|---|
| **StateFSM** | **A** | **B** | **Next StateFSM** |
| S0 | 0 | 0 | S0 |
| S0 | 0 | 1 | S1 |
| S0 | 1 | × | S3 |
| S1 | 0 | × | S2 |
| S1 | 1 | × | S3 |
| S2 | × | × | S4 |
| S3 | 0 | × | S3 |
| S3 | 1 | 0 | S4 |
| S3 | 1 | 1 | S3 |
| S4 | 0 | 0 | S0 |
| S4 | 0 | 1 | S1 |
| S4 | 1 | 0 | S4 |
| S4 | 1 | 1 | S0 |

Table 4–5: State Table for signal *StateFSM*

Both the ST and Boolean function allow the abstraction of dependencies as complex Boolean expressions. For example, the expression "`write_addr_int+1 = read_addr_int`" shown in the $4^{th}$ column of table 4–1 is considered as a unique dependency. Separating each signal in the expression is not required and applying such techniques is desirable in order to facilitate the use and readability of the ST-method.

Multiple columns may also be combined by taking advantage of sub-functions within the ST. Consider tables 4–6 and 4–7 for the function `OUT = (XOR(A,B,C) AND D)`. In table 4–7, signals A, B and C are in the same column and `XOR(A,B,C)` is a sub-function of `OUT`.

| D | A | B | C | OUT |
|---|---|---|---|---|
| 0 | × | × | × | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

Table 4–6: ST for *OUT = (XOR(A,B,C) AND D)*

| D | XOR(A,B,C) | OUT |
|---|---|---|
| 0 | × | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Table 4–7: ST for *OUT = (XOR(A,B,C) AND D)* using a sub-function

Certain signals that share an identical relationship can be merged into a single state table. As an example, consider a grant signal in a round-robin arbiter circuit shown in table 4–8. Table 4–8 shows the ST of a single grant signal (grant_A). Table 4–9 shows the ST for an N-bit signal `grant[0:N-1]` where N is the number of grant signals connected to the arbiter.

| Dependent Signals | | Control Signal |
|---|---|---|
| **req_A** | **selected_master = master_A** | **grant_A** |
| 0 | × | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Table 4–8: Single ST for *grant_A*of a round-robin arbiter

| Dependent Signals | Control Signal |
|---|---|
| **req[selected_master]=1** | **grant[0:N-1]** |
| 0 | grant[0:N-1] = 0 |
| 1 | grant[selected_master]=1 and onehot[grant[0:N-1]] |

Table 4–9: Merged ST for *Grant[0:N-1]* of a round-robin arbiter

The ST of table 4–8 requires 3 assertions for each grant signal; a total of 3N assertions is therefore required for all the grant signals in the round robin arbiter. The ST in table 4–9 on the other hand shows that only 2 assertions are required to cover all of the grant signals in `grant[0:N-1]`.

To summarize, the Boolean function has all of the benefits of the ST method and many additional benefits. The manipulation of a function is easier than the manipulation of state tables, advanced minimization techniques can be applied on functions. Functions work well with software programming and there is no need for ordering as was necessary with state tables (this means that the Boolean function method scales well with the addition of dependencies and works well with software programming)...

# CHAPTER 5
# Writing Sufficient Assertions in Verification

## 5.1 When Have We Written Sufficient Assertions?

### 5.1.1 Assertion Density and Fault Coverage

Assertion density is a measure of the number of assertions per line of code. Many authors consider assertion density to be directly linked with writing enough assertions. With assertion density, the goal is to have uniform assertion density with minimum holes across the entire design [28]. Without sufficient density, the full benefits of assertions are not realized.

Another approach cited involves determining the coverage of a set of assertions against a high-level stuck-at fault model and adding assertions to cover all such faults [5].

### 5.1.2 Writing Sufficient Assertions on a Type by Type Basis

The question, "when have we written sufficient assertions?", can be tackled separately for each type of assertion. In other words, writing sufficient assertions is achieved when each type of assertion has sufficiently been written.

### Sufficient Behavioral Assertions

Obtaining a complete and sufficient set of behavioral assertions can be achieved by implementing the Boolean function method described in Section 4.2 on the desired signals. The control signals that are of interest to designers and verifiers are selected and the other control signals are ignored. This prevents the task of writing sufficient behavioral assertions from becoming exhaustive.

### Sufficient Invariant Assertions

The most common types of invariant assertions are those that are obtainable directly from the behavior of the RTL. For example, "**assert never {full='1' && empty='1'};**" is such an assertion. As was shown in Section 6.4, these assertions can be generated directly from the full and empty functions. Such invariant assertions that are obtainable directly from the Boolean functions are covered by behavioral assertions and may be omitted.

Certain statements found in specification catered towards invariant assertions are in a separate category since the low level RTL is capable of handling the various states and conditions. However, the high level RTL acts as a constraint in such a way that the statements never occur in the described block. For example, the statements in Table 5–1 represent such project-specific invariant assertions.

| | |
|---|---|
| **transfer_b1 [1:0]** | Indicates the current transfer which can be idle, busy or sequential. However, Block_1 does not support the busy state. |
| **burst_b2 [2:0]** | Indicates the burst type. Block_2 only supports single burst. |
| **select_b3 [2:0]** | This output is tied to a constant value of 010 in Block_3 |

Table 5–1: Examples of Project-Specific Invariant Assertions

Project-specific invariant assertions tend to stand out while writing the specification document and the RTL. Writing sufficient amounts of invariant assertions comes down to covering the most important requirements and standards in a given project and this can quickly become subjective due to the nature of invariant assertions. Hence, a complete set of invariant assertions is unrealistic to obtain in view of to the large amounts of invariant relationships that can exist in a practical design.

**Sufficient Temporal Assertions**

Most temporal assertions have well defined purposes. Bounded temporal assertions can be used to ensure fairness, verify that an event occurs within an expected time period, ensure that a protocol is being respected and conduct black box verification. Unbounded temporal assertions can be used to detect starvation and deadlock among other uses.

Writing sufficient temporal assertions is feasible in the case of ensuring fairness, detecting starvation, detecting deadlock and ensuring that protocol requirements are met (protocol requirements tend to be finite by nature). For other cases such as verifying that events occurs within an expected time frame, writing sufficient temporal assertions becomes subjective as a large number of assertions could be

written in these cases. It would be essential to identify and cover the most important scenarios.

### Sufficient Complex Assertions

Complex assertions can take many forms and can be formed with various types of assertions. Since it is easier to determine a purpose for simpler types of assertions, one idea would be to investigate if it was possible to reduce all complex assertions into the simpler sub-types of assertions described in the previous sections.

Table 5–2 provides examples of complex assertions found on the Internet and in related documentatiosn and shows a few attempts at reducing them into simpler assertions.

| | |
|---|---|
| **Complex Assertion** | `{gnt} \|=> {busy[*2]; done};` |
| **Attempt to Simplify** | The assertion can be seen as a combination of 2 behavioral assertions and 1 temporal assertion:<br><br>Behavioral (busy): `{gnt} \|=> {busy};`<br>Temporal (busy): `{!busy; busy} \|-> {[*2]busy; !busy};`<br>Behavioral (done): `{[*2]busy} \|=> {done};` |
| **Complex Assertion** | `{instr = write} \|=> {mem_cs[*2]; !mem_cs}` |
| **Attempt to Simplify** | The assertion in the row above can be seen as 2 behavioral assertions and 1 temporal assertion similarly to the previous assertion. |
| **Complex Assertion** | If A is followed by B, then on the next cycle Y should be followed by Z<br>`(A ##1 B) \|=> (Y ##1 Z);` |
| **Attempt to Simplify** | The above assertion could be a combination of 2 behavioral assertions<br><br>Behavioral(on Y): `{A; B} \|=> {Y}`<br>Behavioral (on Z): `{Y} \|=> {Z};` |
| **Complex Assertion** | `(a ##2 b ##3 (c && d)) \|=> ((e && f) ##2 g);` |
| **Attempt to Simplify** | The above assertion contains temporal assertions, behavioral assertions, invariant assertions and abstract assertions. Due to the abstraction in the above assertion it's hard to say exactly how it should be simplified or if it can be simplified at all. |

Table 5–2: Simplifying Complex Assertions

Many of the complex assertions in Table 5–2 can be regarded as complex abstract assertions. The task of simplifying complex abstract assertions is not always evident.

If a complex assertion can be simplified into simpler subtypes of assertions then it is a advisable to do so. It is easier to determine when sufficient assertions have been written for simple assertions than it is for complex assertions. Simple assertions are

also easier to write, maintain and debug. Many mundane errors may be avoided by writing simpler assertions whereas the complexity may add confusion when reviewing the assertion or determining which part of the complex assertion is incorrect.

Complex and abstract assertions should have well defined purposes such as reducing the total number of assertions, replacing multiple simpler types of assertions or speeding up the simulation time... Otherwise, if both the simple assertions and the abstract or complex assertions are utilized, then the usage of abstract and complex assertions may become redundant.

## 5.2 Who Should Write Assertions?

According to [2], most important assertions are specified by the designer to capture information that otherwise would remain in his or her head. Writing assertions can be performed by both the designer and the verification engineer. Designers write assertions in order to validate the correctness of their implementation, whereas verifiers aim at checking if the entire design is producing the correct response when interfaced with other circuit blocks inside the system. The question of who should write assertions usually depends on the hierarchy in the design where such assertions would be covering. Table 5–3 provides a summary of assigning the different types of assertions to the appropriate group.

| | **Behavioral Assertions** | **Invariant Assertions** | **Project-Specific Invariant Assertions** | **Abstract Assertions** |
|---|---|---|---|---|
| **Designer** | ✓ | ✓ | | |
| **Verifier** | | | ✓ | ✓ |

| | **Bounded Temporal Assertions** | **Unbounded Temporal Assertions** | **Complex Assertions** | **Other Assertions** |
|---|---|---|---|---|
| **Designer** | ✓ | | Low Level hierarchy | Low Level hierarchy |
| **Verifier** | ✓ | ✓ | High level hierarchy | High level hierarchy |

Table 5–3: Who Should Write Assertions Summary

### Behavioral Assertions

Designers should write behavioral assertions on their own blocks as designers usually know how the RTL works better than a verification engineer. Especially with formal verification tools, the designers are best fitted to write behavioral assertions since they can verify their own RTL immediately without having to wait for a test-bench.

### Invariant Assertions

Designers should write invariant assertions on their own blocks (at the local hierarchy) because they usually know better than a verification engineer how the RTL works at the local hierarchy.

Verification engineers on the other hand are responsible for writing invariant relationships that are higher up in the hierarchy. For this reason, verification engineers are also the most likely candidates to write project and environment specific invariant assertions.

**Temporal Assertions**

Designers should write temporal assertions to verify that the RTL works as intended at the local hierarchy. Verifiers on the other hand should write temporal assertions to verify that protocols are respected, to ensure fairness, to detect starvation, and to verify that an event occurs within an expected time period at a higher hierarchy.

**Abstract Assertions**

Either designers or verifiers may write abstract assertions depending on the level of hierarchy which the abstract assertions are being written at. In general, verification engineers will write more abstract assertions as abstract assertions can be placed higher up in hierarchy where the assertions can have a larger cone of influence.

**Complex Assertions**

Similar to abstract assertions, either designers or verifiers may write complex assertions, depending on the level of hierarchy which the assertions are being written at.

# CHAPTER 6
# Optimizing Assertions using Logic Synthesis Techniques

Logic synthesis techniques are widely used when designing RTL to optimize digital circuits and reduce the cost of building chips. Logic synthesis enables the optimization of area, delay, power, etc... [57]. Specifically, the number of implicants and the number of literals of each logic function are directly related to the number of logic gates, and therefor the number of transistors needed which affects the size, area, speed, fan-in, loading, power consumption...

Analysis of the ST and the Boolean function method have shown that advanced forms of logic synthesis may be used to optimize assertions. The aim is to determine if logic synthesis techniques could be used to improve assertion based verification (ABV) in terms of the overall speed of verification.

Assertions are known to consume a lot of resources in both dynamic and formal simulations. Optimized assertions would consume less resources and memory, thereby increasing the number of verification tests that can be run within the limited time period available.

With regards to ABV optimization, and in order to reduce the resources spent on assertion verification and speed up the verification process, the present research work aims at reducing (1) the total number of assertions and (2) the number of

signals being "observed" in each assertion, while (3) guaranteeing a complete and easy to debug assertion set.

## 6.1 Minimizing Assertions by Optimizing the Boolean Function

When optimizing the design of a circuit, designers will try to minimize the number of literals and implicants within various functions. A similar first step has been taken in the present work to minimize (and therefor optimize) the number of assertions.

For example, the Karnaugh map for the full signal in figure 6–1 is used to minimize the full function. The full signal depends on read, write, its previous value and the condition when the write address is equal to the read address.

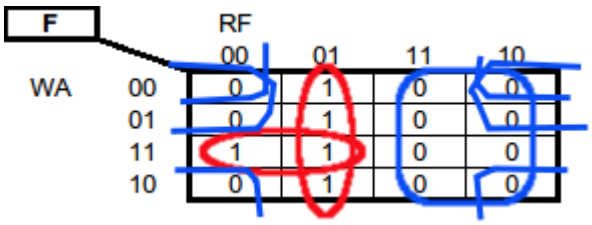



Figure 6–1: Karnaugh map for the Full signal

In all of the Karnaugh maps in this chapter, "F" refers to when the FIFO is full, "E" to when the FIFO is empty, "R" to a read attempt, "W" to write attempt, "O" to overflow, "U" to underflow, "A" refers to the condition when the write address is equal to the read address minus 1, and "B" refers to the condition when the read address is equal to the write address minus 1.

The minimized form for the full function and its inverse function can be obtained from the Karnaugh map. These are found to be:

| |
|---|
| F = R'F + R'WA |
| F' = R + F'W' + F'A' |

Other optimization techniques can also be used to obtain F and F'. The Karnaugh map was chosen in this thesis because of its simplicity and visual clarity. Both the function and the inverse function can be simultaneously obtained directly from the Karnaugh map. The drawback of using Karnaugh maps is that for more complex functions (with more literals), the Karnaugh map breaks down, in which cases other methods should be used to obtain F and F'.

Whichever method is used to obtain F and F', one must verify that they either add up to 1 (tautology) or to a subset of the don't care set in order to insure that all the distinct relevant cases are covered with assertions.

A significant difference between design optimization and assertion optimization using logic synthesis techniques is that, in assertion optimization, both the function and it's inverse are needed for the assertion set to be complete. Whereas in design optimization, either the function or its inverse function is chosen, usually based on which of the two provides the best advantages.

## 6.2 Combining Multiple Assertions

### 6.2.1 Combining Assertions using Common Cubes

There are many situations where multiple assertions can be combined. The simplest case is when the triggers in separate assertions are identical. In other words,

if two or more assertions are observing the same set of cubes then it is possible to merge these assertions together while only affecting the output condition of these assertions.

For example, g and h are two functions with `g = x'y + xz'` and `h = y' + xz'`. The Karnaugh maps for g and h are drawn in Figure 6–2.

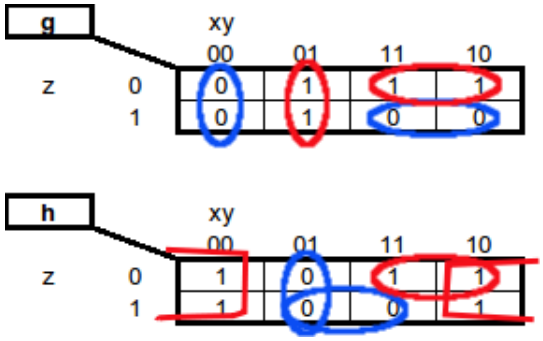



Figure 6–2: Karnaugh Maps for the g and h functions

If we were to write PSL assertions to cover g and h, we would obtain the following 8 assertions:

| | |
|---|---|
| assert always {!x AND y} $\mid\rightarrow$ {g}; | assert always {x AND !z} $\mid\rightarrow$ {h}; |
| assert always {x AND !z} $\mid\rightarrow$ {g}; | assert always {!y} $\mid\rightarrow$ {h}; |
| assert always {!x AND !y} $\mid\rightarrow$ {!g}; | assert always {!x AND y} $\mid\rightarrow$ {!h}; |
| assert always {x AND z} $\mid\rightarrow$ {!g}; | assert always {y AND z} $\mid\rightarrow$ {!h}; |

Table 6–1: g and h Assertions

From the Karnaugh maps of Figure 6–2, one notices that the cube x'y is covered by both the g and !h functions and the cube xz' is covered by both the g and h functions. This allows us to merge the affected assertions together to obtain:

```
assert always {!x AND y} |→ {g AND !h};
assert always {x AND !z} |→ {g AND h};
```

This simple operation reduces the total number of assertions required to cover g and h from 8 to 6.

### 6.2.2 Merging Multiple Karnaugh Maps

The previous section provides the insight that assertions can be merged but is limited by the requirement of pre-existing common cubes. During the search for a more generalized method of combination assertions it was found that merging Karnaugh can reduce the total number of assertions.

#### Merging the Karnaugh Maps of g and h

To create such a merged Karnaugh map for g and h, one simply takes the values in the Karnaugh map of h and adds them to the Karnaugh map of g. The result is the Karnaugh map shown in Figure 6–3. It is worth noting that the values inside the Karnaugh map of Figure 6–3 are now double digit (referring to the possible output combinations of g and h). The cubes in a merged Karnaugh map are selected based on the four different values that {g,h} can take. From Figure 6–3, the merged Karnaugh of gh contains a total of 5 cubes, which reduces the total number of assertions from 8 to 5.

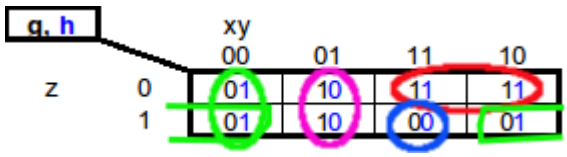



Figure 6–3: Merged Karnaugh map for 'g' and 'h'

The 5 assertions obtainable from the merged Karnaugh map of g and h are:

| |
|---|
| assert always {!x AND !y} \|→ {!g AND h}; |
| assert always {!x AND y} \|→ {g AND !h}; |
| assert always {x AND !z} \|→ {g AND h}; |
| assert always {x AND y AND z} \|→ {!g AND !h}; |
| assert always {!y AND z} \|→ {!g AND h}; |

Table 6–2: List of Assertions Obtained When Merging the g and h Karnaugh Maps

From a mathematical point of view, merging Karnaugh maps is equivalent to calculating $2^n$ different cases where n is the number of Karnaugh maps to be merged. In our case merging Karnaugh maps is the equivalent to obtaining all the logic functions g.h, g'.h, g.h' and g'.h' as shown in Table 6–3.

| |
|---|
| gh = (x'y + xz')(y' + xz') = xy'z' + xz' = xz' |
| g'h = (x'y' + xz)(y' + xz') = x'y' + xy'z = x'y' + y'z |
| gh' = (x'y + xz')(x'y + y'z) = x'y +x'yz = x'y |
| g'h' = (x'y' + xz)(x'y + y'z) = xyz |

Table 6–3: Mathematical Equivalence of Merging Karnaugh Maps

The equivalence of the two methods can be seen by comparing the values obtained mathematically in Table 6–3 to the Karnaugh map cubes in Figure 6–3. As

a note !x and x' are used interchangeably and mean the same thing in this thesis. Similarly AND and && are also used interchangeably.

In a general case, the functions to be merged must be mapped to a common Karnaugh map containing all the variables in each function. In the example above, both g and h depended on the same three variables: x, y and z. Functions in general may not have all of their variables in common as is the case with the FIFO signals.

### Case Study: Merging Karnaugh Maps for the FIFO Signals

To gain further insight into the merit of merging Karnaugh maps, the Karnaugh maps of the FIFO signals {F,E,O,U} will be merged. The individual (non-merged) Karnaugh maps for these signals are sketched in Figures 6–4, 6–5 and 6–6.

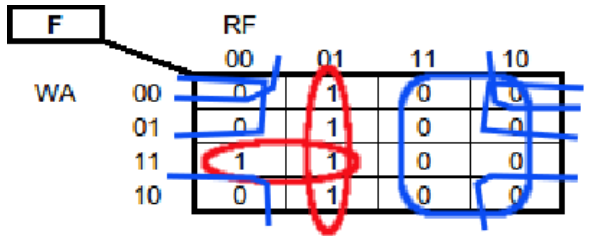



Figure 6–4: Karnaugh map for Full

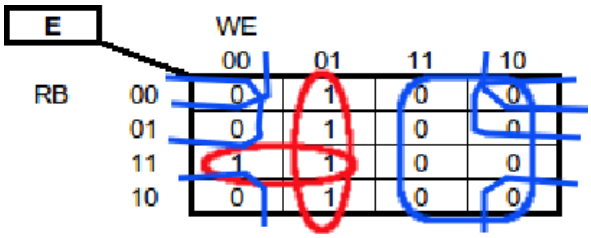



Figure 6–5: Karnaugh map for Empty

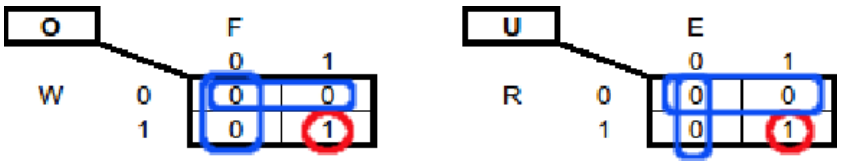



Figure 6–6: Karnaugh maps for Overflow and Underflow

From the Karnaugh maps we can obtain the minimal functions and inverse functions for F, E, O and U. These are given in Table 6–4.

| | |
|---|---|
| F = R'F + R'WA | F' = R + F'W' + F'A' |
| E = W'E + W'RB | E' = W + E'R' + E'B' |
| O = FW | O' = W' + F' |
| U = ER | U' = R' + E' |

Table 6–4: FIFO Functions and Inverse Functions

The number of assertions required to cover each signal is equal to the number of cubes within each Karnaugh map including both the function and inverse function cubes. From the Karnaugh maps in Figures 6–4, 6–5 and 6–6, "F" requires 5 assertions, "E" requires 5 assertions, "O" requires 3 assertions and "U" requires 3 assertions for a total of 16 assertions. These 16 assertions provide us with a complete set of assertions to verify F, E, O and U. Without combining assertions this is also the minimal complete set of non-redundant behavioral assertions.

### Merged Karnagh Map for {F,O}

The Karnaugh maps of F and O are merged in Figure 6–7 and the result produces 8 assertions (8 distinct cubes). F is dependent on the 4 signals {R,F,W,A} whereas O is only dependent on the 2 signals {F,W} . In order to successfully merge the Karnaugh maps, all of the dependent signals must be in the merged Karnaugh map.

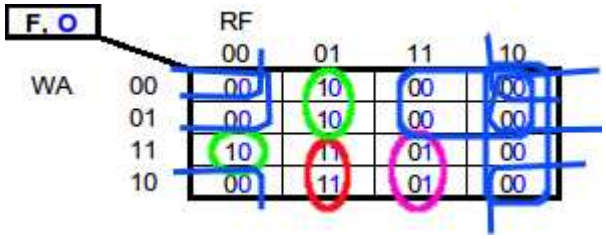



Figure 6–7: Merged Karnaugh map for Full and Overflow

In this particular scenario, there is no benefit in merging the Karnaugh maps for F and O as the number of assertions was not reduced (F already required 5 assertions and O required 3 assertions which adds up to a total of 8 assertions).

**Merged Karnaugh Map for {F,E}**

Next, the Karnaugh maps of F and E are merged, as shown in Figure 6–8, resulting in 11 assertions (11 distinct cubes). Conversely in this case, merging Karnaugh maps produces more assertions than previously obtained (F required 5 assertions and E also required 5 assertions for a total of 10 assertions). This result demonstrates that merging Karnaugh maps does not always provide a smaller number of assertions especially when many of the dependencies are different.

*For visual clarity, only the output values corresponding to a logical 1 are shown while the remaining logical 0 values have been dropped. Each logical 1 is colored according to which output it corresponds to (Black for "F", red for "E", blue for "O" and green for "U").*

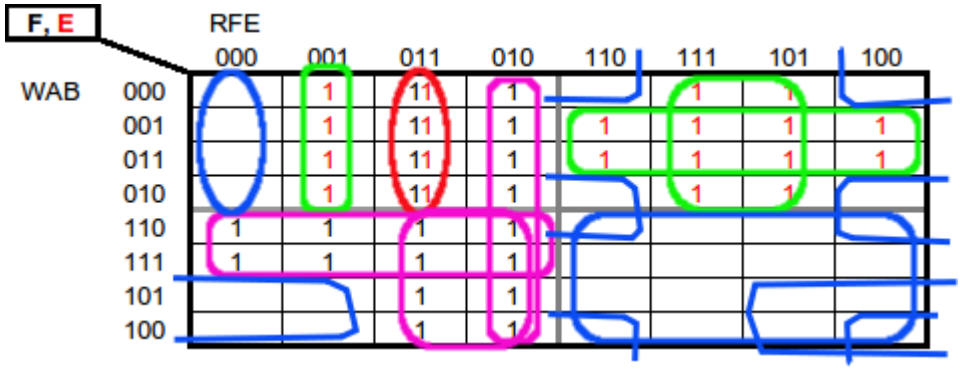



Figure 6–8: Merged Karnaugh map for Full and Empty

### Merged Karnagh Map for {F,U}

Next, the Karnaugh maps for F and U are merged in Figure 6–9 and the results produces 6 assertions, a reduction of 2. As without merging Karnaugh maps, F is covered by 5 assertions and U is covered by 3 assertions for a total of 8 assertions.

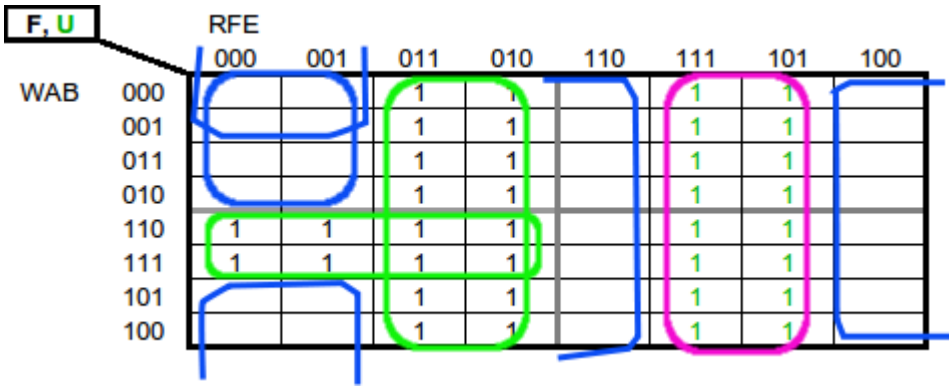



Figure 6–9: Merged Karnaugh Map for Full and Underflow

Due to the symmetry between {F and E} and {O and U}, the Karnaugh maps for EU and EO were not shown as they are identical to the Karnaugh maps of FO and FU in Figures 6–7 and 6–9.

**Merged Karnagh Map for {F,E,O,U}**

Finally, by merging the Karnaugh maps of the 4 functions {F,E,O,U} in a single merged Karnaugh map as shown in Figure 6–10, we obtain 14 assertions, a reduction of 2 assertions.

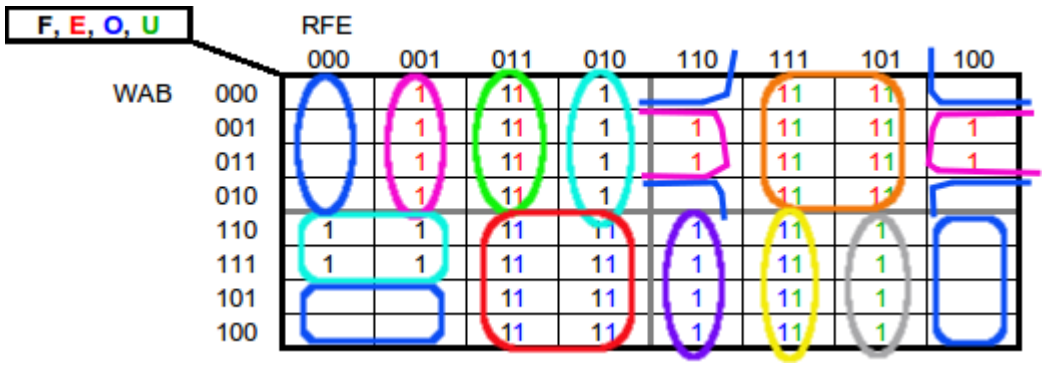



Figure 6–10: Merged Karnaugh Map for Full, Empty, Overflow and Underflow

It is interesting to note that merging the Karnaugh maps of FU and EO would reduce the total number of assertions from 16 to 12 for the synchronous FIFO. While merging all 4 Karnaugh maps as was done in Figure 6–10 provided a lesser benefit in terms of the total number of assertions than selectively merging the Karnaugh maps of FU and EO.

An overview of the advantages and drawbacks of the different methods to combine assertions outlined in this chapter is provided in Section 6.3.2.

### 6.2.3 Heuristic Techniques

For multi-level synthesis and for large number of inputs (inputs $>$ 10), the Karnaugh map and the mathematical brute force techniques do not work well. In

such scenarios, heuristic techniques could alleviate the computational burden while at the same time providing a minimal and easy way to debug the set of assertions.

**Example of the Heuristic Techniques Applied**

To provide an insight for the use of heuristics, three functions A, B and C were examined:

```
A = xy' + yw'
B = xy(z' + w') + zw(x' + y')
C = xy + yz + zx + zw
```

Table 6–5: Minimized Functions for A, B and C

The Karnaugh maps for the functions A, B and C as well as the merged Karnaugh map ABC are shown in Figure 6–11. The separate Karnaugh maps for A, B, C are covered by a total of 20 assertions (4 for A, 9 for B, and 7 for C), while the merged Karnaugh map is only covered by 11 assertions.

The heuristic methods steps are shown in Figures 6–12, 6–13 and 6–14 and a description of its application is provided below:

1. Detect and combine common cubes. This step was done on the x'z' and y'z' cubes that are circled in gray in Figure 6–12.
2. Determine if one cube containing another can be reduced (not split) so that both cubes can be covered by the same assertion while still providing complete coverage. This step is shown in Figure 6–13 on the xy cube in the Karnaugh map of C which was reduced to the xyz' cube.
3. Determine if a large cube in one Karnaugh map, fully covering multiple cubes in another Karnaugh map, can be split so that the assertions can be combined

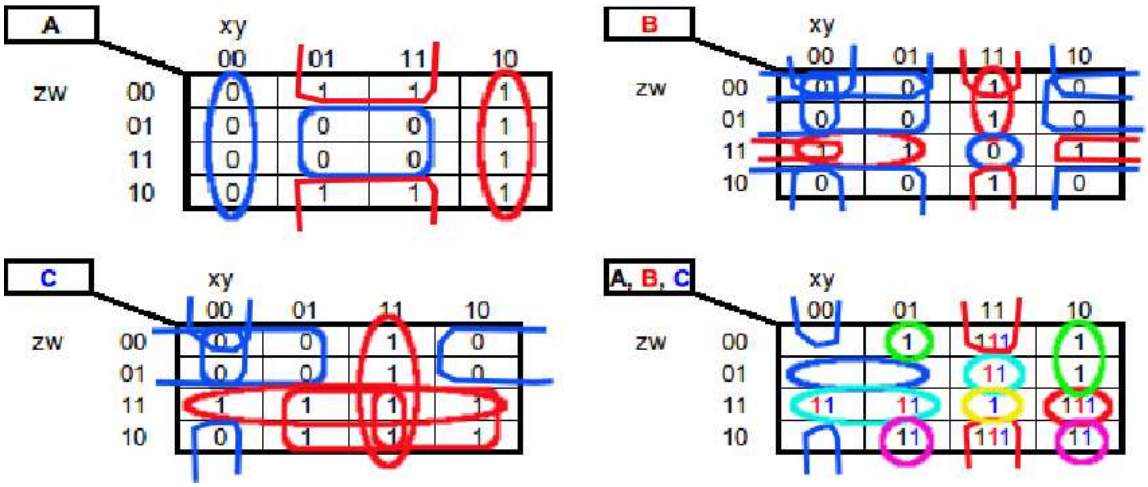



Figure 6–11: Karnaugh Maps for A, B and C as Well as the Merged Karnaugh Map

using common cubes as described in step 1. This step is shown in Figure 6–14 on the yw' cube which was split into the yw'x' and yw'x cubes.

4. Determine if distinct cubes sharing a common intersection can be reduced simultaneously to a common cube. This step produced the xy'zw' cube in Figure 6–14.
5. Remove redundant cubes if any were found and re-expand linked cubes if possible.

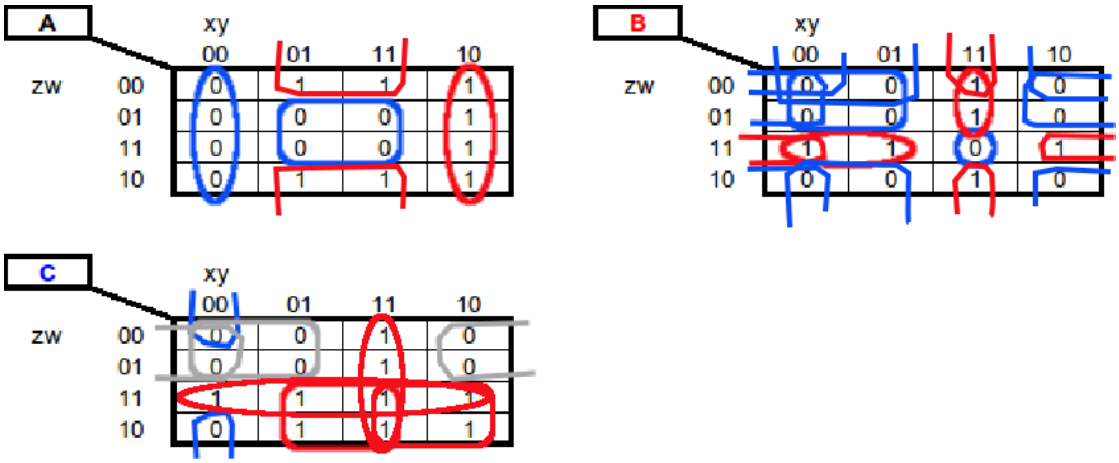



Figure 6–12: Identifying Common Cubes in Karnaugh Maps

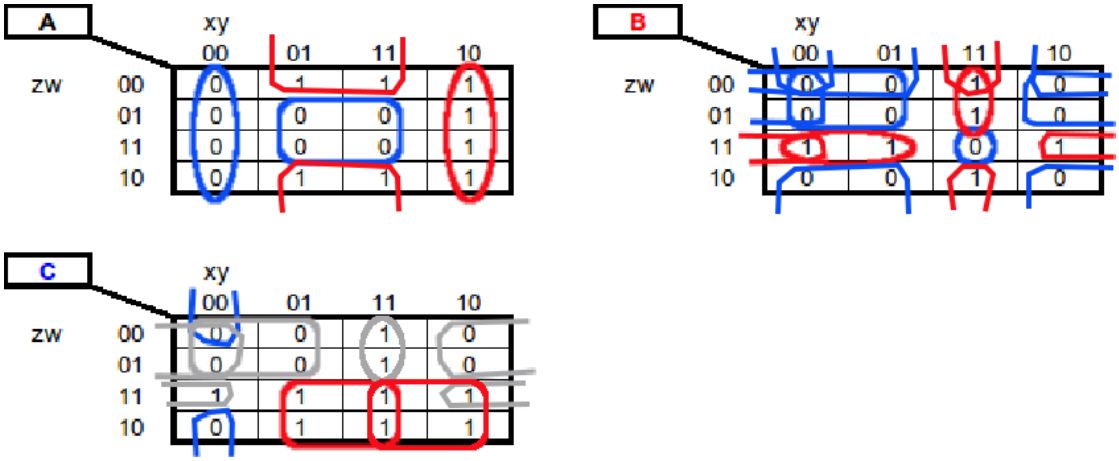



Figure 6–13: Identifying Cube Containment in Karnaugh Maps

It is important to verify that every change, or every group of changes, decreases the number of assertions, otherwise it must be reverted back. The heuristic steps described above share a resemblance to those found in EXPRESSO and other well known heuristics logic minimizers (particularly the expand, reduce and irredundant steps).

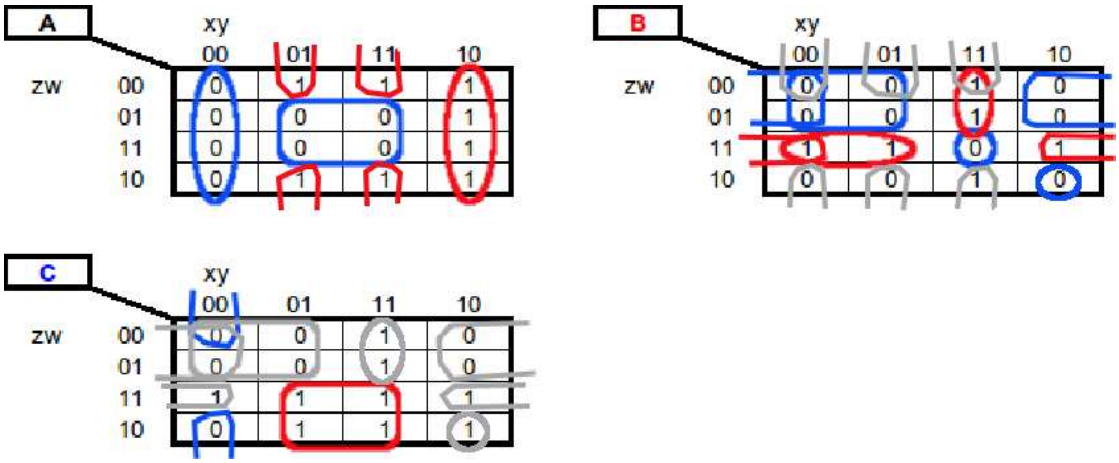



Figure 6–14: Applying Multiple Reductions on Cubes

**Comparison of the Assertions Generated by the Different Techniques**

The assertions obtained from the minimized functions, the merged Karnaugh maps and the heuristic techniques for the signals {A,B,C} can be found in tables 6–7 and 6–8. The results in Table 6–7 and 6–8 show that the minimized functions require a total of 20 assertions, the merged Karnaugh maps requires only 11 assertions while the heuristic techniques reduces the number of assertions to 14.

| Assertions obtained from the minimized functions of A, B and C (see Figure 6–11 for their separate Karnaugh maps) | Assertions obtained from the merged Karnaugh map of ABC drawn in Figure 6–11 |
|---|---|
| {x & !y} \| => {A};<br>{y & !w} \| => {A};<br>{!x & !y} \| => {!A};<br>{y & w} \| => {!A};<br><br>{x & y & !z} \| => {B};<br>{x & y & !w} \| => {B};<br>{z & w & !x} \| => {B};<br>{z & w & !y} \| => {B};<br>{!x & !z} \| => {!B};<br>{!y & !w} \| => {!B};<br>{!x & !w} \| => {!B};<br>{!y & !z} \| => {!B};<br>{x & y & w & z} \| => {!B};<br><br>{x & y} \| => {C};<br>{z & w} \| => {C};<br>{y & z} \| => {C};<br>{x & z} \| => {C};<br>{!x & !z} \| => {!C};<br>{!y & !z} \| => {!C};<br>{!x & !y & !w} \| => {!C}; | (RED Cubes)<br>{x & y & !w} \| => {A & B & C};<br>{x & !y & w & z} \| => {A & B & C};<br><br>(Cyan Cubes)<br>{!x & w & z} \| => {!A & B & C};<br>{x & y & !z & w} \| => {!A & B & C};<br><br>(Fuchsia Cubes)<br>{!x & y & z & !w} \| => {A & !B & C};<br>{x & !y & z & !w} \| => {A & !B & C};<br><br>(Green Cubes)<br>{x & !y & !z} \| => {A & !B & !C};<br>{!x & y & !z & !w} \| => {A & !B & !C};<br><br>(Yellow Cube)<br>{x & y & z & w} \| => {!A & !B & C};<br><br>(Blue Cubes)<br>{!x & !y & !w} \| => {!A & !B & !C};<br>{!x & !z & w} \| => {!A & !B & !C}; |
| Total of 20 Assertions | Total of 11 Assertions |

Table 6–7: ABC Assertion Table Comparison 1

| Assertions obtained from the heuristic techniques applied to {A,B,C} as demonstrated in Figure 6–14. |
|---|
| {x & !y} \| => {A};<br>{!x & !y} \| => {!A};<br>{y & w} \| => {!A};<br><br>(Common Cubes btw A and B)<br>{!x & y & !w} \| => {A & !B};<br>{x & y & !w} \| => {A & B};<br><br>(Common Cubes btw B and C)<br>{!x & !z} \| => {!B & !C};<br>{!y & !z} \| => {!B & !C};<br>{!x & !y & !w} \| => {!B & !C};<br>{x & y & !z} \| => {B & !C};<br>{!y & z & w} \| => {B & C};<br>{x & !y & z & !w} \| => {!B & C};<br><br>{!x & z & w} \| => {B};<br>{x & y & w & z} \| => {!B};<br><br>{y & z} \| => {C}; |
| Total of 14 Assertions |

Table 6–8: ABC Assertion Table Comparison 2

The assertions obtained by merging Karnaugh maps require the verifier to state explicitly the value of each output in each assertion as shown in Table 6–7. This is one of the main drawbacks of merging Karnaugh maps. Although the number of assertions is minimized, the assertions themselves become more crowded with literals and this could render the debugging of these assertions more difficult.

**Heuristic Techniques Applied to FEOU**

The result of the heuristic technique applied to FEOU are given in Figure 8–1 in Appendix A. A list of the assertions for FEOU obtained by implementing all of the various techniques described in this chapter can be found in the tables of Appendix B.

### 6.2.4 Combining Assertions using Temporal Operators

By analyzing the inverse function of overflow `!O=!W+!F`, it can be noticed that whenever F=0 then O=0 at the next clock cycle. As assertions for F already exist, the number of assertions can be reduced by combining the assertions of !O with that of !F. The assertions affected are shown in Table 6–9.

| |
|---|
| The 4 original assertions affected by this change are<br><br>{R} \| => {!F}<br>{!F & !W} \| => {!F}<br>{!F & !A} \| => {!F}<br><br>{!F} \| => {!O}<br><br>By using temporal operators, we can reduce the number of assertions resulting in the following 3 assertions<br><br>{R} \| => {!F; !O}<br>{!F & !W} \| => {!F; !O}<br>{!F & !A} \| => {!F; !O} |

Table 6–9: Using Temporal Operators to Combine Assertions

Applying this procedure to both O' and U' will reduce the total number of assertions by 2. However, this only works in the cases where either the function

depends on a cube of a single variable as was the case with overflow or if a cube of the function happened to appear in the output of a merged assertion.

### 6.2.5 State Variables as Karnaugh Map Outputs

An interesting modification to the techniques described above that takes advantage of the abstract nature of assertions is to allow the Karnaugh map values to be dependent on the past state variables. This modification is demonstrated in the Karnaugh maps of Figure 6–15.

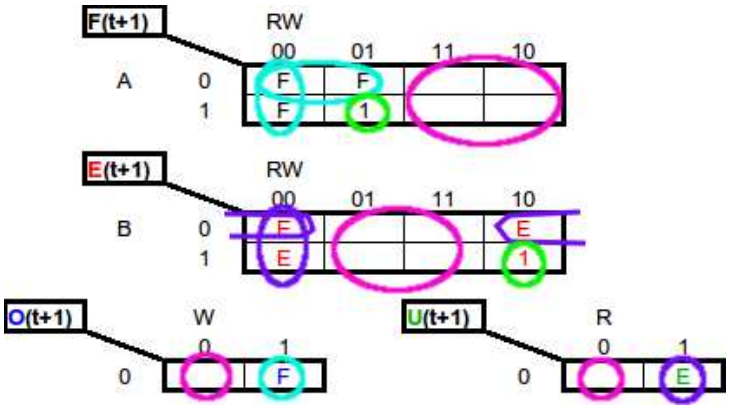



Figure 6–15: Karnaugh maps using dependent outputs for F,E,O and U

In Figure 6–15, the outputs of the Karnaugh maps can either be 1, 0 or a previous value of F and E. It can be noticed that the Karnaugh maps of Figure 6–15 are smaller than their equivalent Karnaugh maps in Figures 6–4, 6–5 and 6–6. Allowing the Karnaugh map output values (values inside the Karnaugh map grid) to be dependent on previous values of F and E, reduces the need to have F and E as one of the grid variables (instead of having a 4x4 Karnaugh map for F in Figure 6–4, a 4x2 Karnaugh map for F(t+1) in Figure 6–15 is achieved).

Such a change reduces the number of assertions to 4 for E and F and 2 for O and U, for a total of 12 assertions in comparison with 16 assertions without taking advantage of any further modification. The assertions obtained by using state variables as Karnaugh map outputs are shown in Table 6–10.

| |
|---|
| {!R & !A} \| => {$*stable*(*F*)}; |
| {!R & !W} \| => {$*stable*(*F*)}; |
| {!R & W & A} \| => {F}; |
| {R} \| => {!F}; |
| |
| {!W & !B} \| => {$*stable*(*E*)}; |
| {!W & !R} \| => {$*stable*(*E*)}; |
| {!W & R & B} \| => {E}; |
| {W} \| => {!E}; |
| |
| {W} \| => {*O* == $*past*(*F*)}; |
| {!W} \| => {!O}; |
| |
| {R} \| => {*U* == $*past*(*E*)}; |
| {!R} \| => {!U}; |

Table 6–10: Using state variables as Karnaugh map outputs to minimize assertions

**Merging Karnaugh Maps with State Variable Outputs**

The Karnaugh maps in Figure 6–15 can be merged into a single FEOU(t+1) Karnaugh map as shown in Figure 6–16. The total number of assertions for FEOU in the merged Karnaugh map is found to be 6. This is a huge reduction in the total number of assertions from the original assertion set which required 16 assertions.

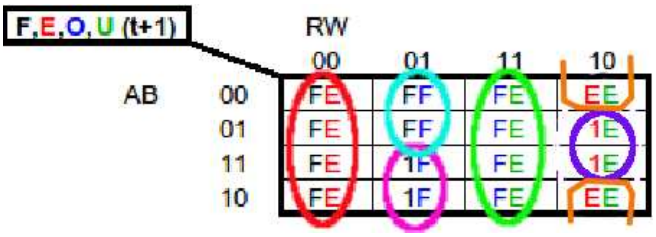



Figure 6–16: Merged Karnaugh Maps Using Dependent Outputs for F,E,O and U

### Merging Selective Karnaugh Maps with State Variable Outputs

For completeness, the merged Karnaugh maps for FU, FO and FE with past state variables are shown in Figure 6–17.

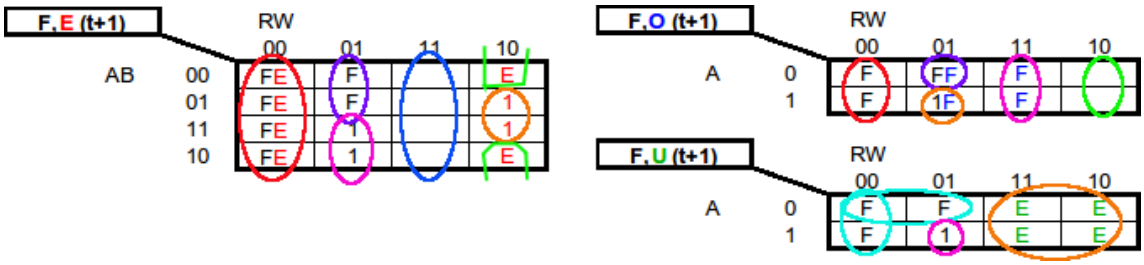



Figure 6–17: Merged Karnaugh Maps Using Dependent Outputs for FE, FO and FU

### Heuristics Applied on Karnaugh Maps with State Variable Outputs

Heuristic techniques can also be applied on the Karnaugh maps in Figure 6–15 and the result are shown in Figure 6–18. The figure depicts a total of 7 assertions to cover F,E,O and U. Once again this is a substantial reduction in the total number of assertions from the original assertion set which required 16 assertions, down to 7.

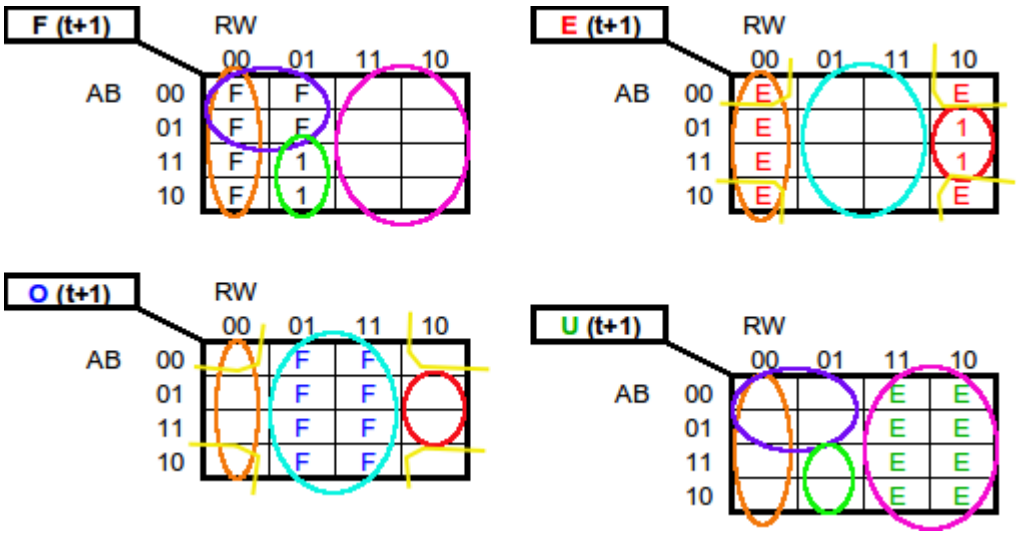



Figure 6–18: Merged Karnaugh Maps Using Dependent Outputs and Heuristics

## 6.3 Overview of the Various Methods to Used Combine Assertions

### 6.3.1 Assertions Obtained from the Various Methods

The assertions obtained from all of the different methods described in this chapter can be found in Tables 8–2, 8–3, 8–4, 8–5 and 8–6 in appendix B.

### 6.3.2 Comparison of the Various Methods

A summary of the total number of assertions obtained by applying the logic synthesis techniques described in the sections above is shown in Table 6–11.

| Method applied | Total assertions obtained |
|---|---|
| Boolean function minimization (default) | 16 assertions |
| Heuristic techniques | 14 assertions |
| Merging Karnaugh maps | 14 assertions |
| Boolean function minimization in addition to temporal operators | 14 assertions |
| Boolean function minimization in addition to $past() and $stable() | 12 assertions |
| Heuristic techniques in addition to $past() and $stable() | 7 assertions |
| Merging Karnaugh maps in addition to $past() and $stable() | 6 assertions |

Table 6–11: Total Number of Assertions Obtained by Each Method

The advantages and drawbacks of each of the described methods used to optimize assertions are discussed in Table 6–12 below.

| | Advantages | Drawbacks |
|---|---|---|
| **Minimized Functions** | To obtain minimized assertions, one simply has to determine the function and the inverse function and verify they add up to 1 or a subset of the don't care set.<br><br>Minimized assertions is by far the simplest method to use that guarantees completeness of the behavioral assertion set with respect to the signals being covered.<br><br>Minimized assertions have the benefit of being the simplest assertions among the various methods in terms of easiness to write and debug. | The drawback of minimized assertions is that they do not take advantage of the possibilities to combine assertions and thereby reduce the assertion count. |
| **Merged Karnaugh Maps** | In most tests, the merged Karnaugh maps provided the minimal number of assertions. | The assertions generated from merged Karnaugh maps must contain the output value of all of the functions even if their value is 0. However, this complicates the readability of the assertions.<br><br>Some of the cubes in a merged Karnaugh map have been so significantly reduced that many of the literals were not needed for most outputs. |

Table 6–12: Comparison of the Various Optimizations

| | Advantages | Drawbacks |
|---|---|---|
| | Merging Karnaugh maps also enables the determining of invariant assertions and unreachable states as will be demonstrated in Chapter 6.4. | For multi-level synthesis and for large number of inputs (inputs>10), the Karnaugh map and the mathematical brute force techniques do not work well.<br><br>The merged Karnaugh map does not always provide fewer assertions, sometimes it may add more assertions. |
| **Heuristics** | For large number of inputs, when the mathematical brute force techniques do not work well, heuristics techniques significantly alleviate the computational burden.<br><br>Although Karnaugh maps were provided for visual clarity, heuristics techniques do not require these maps. Only the functions are needed. | Heuristics require software programs and can be difficult to compute by hand. |

Table 6–12: Comparison of the Various Optimizations

| | Advantages | Drawbacks |
|---|---|---|
| | Heuristics provides the best overall optimization when looking at multiple factors such as the minimal number of assertions required, the readability of the assertions and the speed performance in ABV simulations...<br><br>The heuristics used to optimize assertions share many similarities with existing heuristics used in logic minimization. | In most cases, merged assertions provided fewer assertions than heuristics.<br><br>Results may vary from one heuristic software implementation to another. |
| **Temporal Operators** | Can theoretically be used with all the methods to reduce the number of assertions even further. | There are 2 requirements that must be met before temporal operators may be used to minimize the behavioral assertion set. These requirements drastically reduce the usage of the temporal operator method.<br><br>The first requirement is that the output in one assertion must be identical to the input of another assertion. The second requirement is that all possible ways to obtain such output must be covered by the available assertion set. |

Table 6–12: Comparison of the Various Optimizations

| | Advantages | Drawbacks |
|---|---|---|
| **Previous State Variables** | Applicable to minimized functions, merged Karnaugh maps and heuristics.<br><br>Simplifies the Karnaugh maps and the computation required to obtain assertions.<br><br>Improves the efficiency of all of the methods and reduces the number of assertions even further.<br><br>Takes advantage of the abstract and complex nature of assertions. | The assertions obtained are more abstract and may be more difficult to debug. |

Table 6–12: Comparison of the Various Optimizations

## 6.4 Generating Invariant Assertions

### 6.4.1 Obtaining Invariant Assertions from the Merged Karnaugh Map

The maximum number of assertions obtainable from a single Karnaugh map is related the size of the Karnaugh map. For example in a 4x4 Karnaugh map, the maximum number of assertions amounts to 16, that is 1 assertion for each different input combination. In general, the maximum number of assertions for a single Karnaugh map is $2^n$ where n is the number of inputs that the Karnaugh map depends on.

In simple Karnaugh maps, the possible output values is usually either 0, 1 or X. In a merged Karnaugh map and ignoring the don't care cases for simplicity, the number of possible outputs values is $2^m$ where m is the number of functions that were merged.

In the case of the Karnaugh map for FEOU in Figure 6–10, there is a maximum of $2^6$ or 64 assertions (FEOU depends on 6 inputs which are RFE and WAB), and the output of FEOU can only contain $2^4$ or 16 different output combinations.

By inspecting the output values that are obtained in the Karnaugh maps, one can also deduce by inversion all the output patterns that do not occur. Such output patterns (that do not occur) can be regarded as invariant assertions.

For example, from the Karnaugh map in Figure 6–10, the output values that can occur in (F,E,O,U) are {0000, 0001, 0010, 0011, 0100, 0101, 1000, 1010, 1100}, while by inversion the output values that can't occur are {0110, 0111, 1001, 1011, 1101, 1110, 1111}. To obtain a complete set of invariant assertions affecting these outputs, one could construct all prime implicants of the output values that don't occur by using the Quine-McCluskey algorithm as demonstrated in Figure 6–19.

From this result we obtain two prime invariant assertions, which are {E=1 & O=1} and {F=1 & U=1}. Obtaining this result mathematically, without the use of the Karnaugh map, comes down to calculating the values of all the various output combinations {F'E'O'U', F'E'O'U, F'E'OU', ...} and determining the empty sets. Once the empty sets are found, they can be minimized to obtain the prime implicants using the Quine-McCluskey algorithm as described above.

| FEOU | Step1 | Step2 | Step3 |
|---|---|---|---|
| 0110 | 011* | *11* | *11* |
| 0111 | *110 | *11* | |
| 1001 | *111 | | |
| 1011 | 10*1 | 1**1 | 1**1 |
| 1101 | 1*01 | 1**1 | |
| 1110 | 1*11 | | |
| 1111 | 11*1 | | |
| | 111* | | |

Figure 6–19: Prime Invariant Assertions using Quine-McCluskey

As an example, FE'O'U should be empty according to the Karnaugh map. Verifying this mathematically using the values of Table 6–4 yields:

```
FE'O'U = (R'F + R'WA)(W + E'R' + E'B')(W' + F')(ER)
       = (R'F + R'WA)(ER)(W + E'R' + E'B')(W' + F')
       = (0 + 0)(W + E'R' + E'B')(W' + F')
       = 0
```

The task of obtaining invariant assertions mathematically can be further simplified if one restricts the order of the invariant assertions to 2. Invariant assertions of order 2 only involve two signals at a time and they are the type of invariant assertions most commonly found in verification. In such a case an attempt to determine the existence of invariant relations between FE, FO, FU, EO, EU and OU is carried out. The number of invariant assertions to be found is now $(m-1)!$ where m is the number of functions.

The fact that invariant assertions can be obtained directly from the functions (which are covered by behavioral assertions) seems to pose a new question to verifiers

writing assertions which is: are invariant assertions of this type redundant? The reasoning is that the functions are already fully covered by behavioral assertions, and the obtained invariant assertions are covering the relationship that already exists within the behavioral assertions. It might turn out to be that invariant assertions are more useful as a means to restrict design from a planning point of view rather than to verify the operation from a functional point of view which is the role of behavioral assertions.

### 6.4.2 Determining Unreachable States

By looking closely at the Karnaugh map of FEOU in Figure 6–10, it is observed that the state {F=1 and E=1} is only reachable from its own state {F=1 and E=1}. Knowing that the reset signal sets F to 0 and E to 1, it can be deduced that the state {F=1 and E=1} is unreachable. Invariant assertions can be written here to verify the proper operation of the RTL and that such unreachable states are never observed.

The state diagram of FEOU shown in Figure 6–20 and obtained from the merged FEOU Karnaugh map in Figure 6–10 shows that the states {F=1 and E=1} and {O=1 and U=1} are unreachable. This allows the removal of 2 behavioral assertions from the list and be replaced with 2 invariant assertions. This also has the added benefit of simplifying behavioral assertions by removing unneeded literals in the remaining assertions. The k-map is show in Figure 6–21:

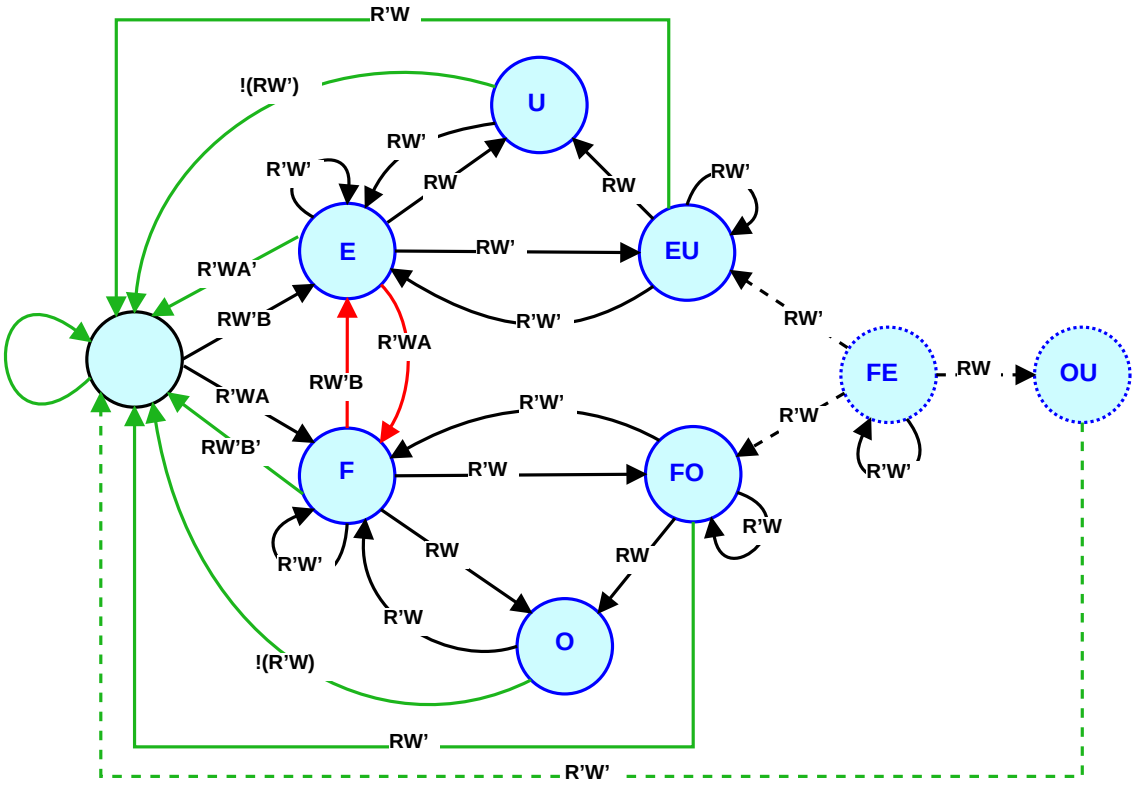



Figure 6–20: State Diagram with Unreachable States

The total number of behavioral assertions has now been reduced to 12 by using the unreachable states. However, it still needs to be determined whether this provides any meaningful gains since 2 invariant assertions have been added.

The Karnaugh map could be further modified by adding the restriction that the condition {A=1 & B=1} never occurs (which means that the FIFO size is never equal to 3).

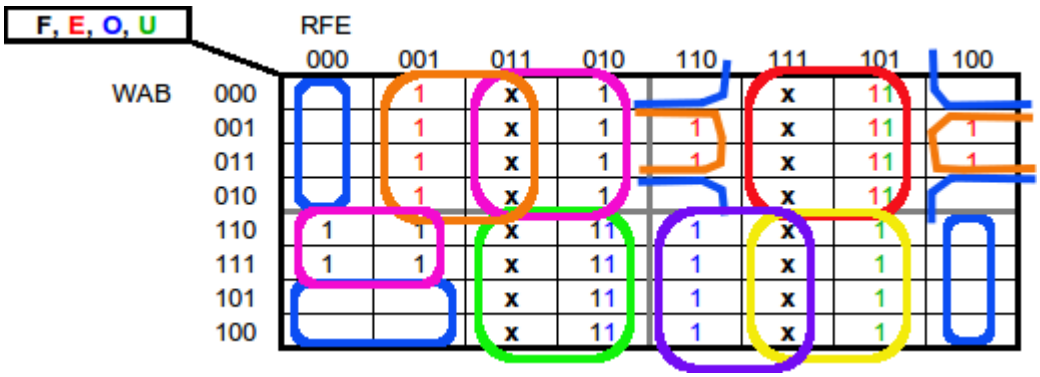



Figure 6–21: Merged Karnaugh Map After Removing Unreachable States for FEOU

# CHAPTER 7
# IFV Results and Time Resources

## Experimental Setup

Once the assertions were written and optimized in a theoretical framework, the next step was to test the optimized assertions in a practical setting to see if they do indeed provide the significant gains that we were hoping for.

To do this, assertions were written for an industrial Synchronous FIFO Controller provided by PMC-Sierra and were verified using the incisive simulator IEV 10.20-s118 by Cadence Design Systems.

The tests described in this chapter were performed on an HP workstation XW4100 with the following characteristics: Intel Pentium 4, CPU MHz: 2793.154, cache size: 512 KB, bogomips: 5589.08.

## 7.1 Varying the length of the data structure

In the test described in Table 7–1, an assertion set consisting of 51 assertions was used to verify the proper behavior of a synchronous FIFO controller. These 51 assertions were written by PMC for their FIFO blocks and reused in the tests to gather data from industrial designs. The FIFO size of the controller was increased in each test and the total time to verify all assertions in IFV was noted.

In all of the tests in Table 7–1, the effort was set to 30 min. "Effort" refers to the maximum amount of time that IFV will spend per assertion. If an assertion takes longer than 30 min to verify, then IFV will declare it "explored" (it's not a pass nor a fail, it just means incomplete) and move on to another assertion.

| FIFO_SIZE | Time to verify all assertions | status |
|---|---|---|
| 3 | 23 seconds | All assertions passed. |
| 8 | 1 min, 22 seconds | All assertions passed. |
| 16 | 8 min, 50 seconds | All assertions passed. |
| 32 | 2 hours, 2 min, 3 seconds | 2 assertions were explored. |
| 64 | 8 hours, 9 min, 27 seconds | 11 assertions were explored. |

Table 7–1: Increasing the FIFO Size with effort=30min and a total of 51 assertions

The results from Table 7–1, show that increasing the FIFO size will drastically affect the time to verify the assertions. With a FIFO size of 3, it takes only 23 seconds to verify all 51 assertions, while increasing the FIFO size to 64 will result in the verification tool requiring over 8 hours to completely verify the same assertion set

This means that minimizing the length of all data structure is essentially in formal verification and will significantly speed up the verification of the assertion set. The time needed for IFV to finish the same set of assertions grows exponentially as the size of the data grows. This test shows why formal verification is not well suited to verify datapath intensive blocks.

## 7.2 Handful of assertions could consume the vast majority of resources

The test in Table 7.2 was performed by keeping the FIFO_SIZE stable to 32 while only modifying the effort.

| Effort | Time to verify all assertions | status |
|---|---|---|
| 5 min | 1 hours, 9 min, 35 seconds | 5 assertions were explored. |
| 30 min | 2 hours, 2 min, 3 seconds | 2 assertions were explored. |
| 50 min | 2 hours, 42 min, 35 seconds | 2 assertions were explored. |

Table 7–2: Increasing the effort with FIFO Size set to 32 with a total of 51 assertions

In the above test, 5 assertions (Out of a total of 51 assertions) took longer than 5 minutes and 2 assertions took longer than 50 minutes. This test shows that sometimes it is only a handful of assertions that are taking up the large amount of time.

## 7.3 CPU Time for Each Method

The test in Tables 7–3 to 7–5 compare the different methods to optimize assertions that were described in Chapter 6. The total amount of time to verify the equivalent set of assertions (but optained bu each different method) is noted and the test is repeated by changing the FIFO size.

| Method used to obtain assertions | Total number of assertions | FIFO Size | Engine used | Total CPU time (sec) |
|---|---|---|---|---|
| Minimized function | 16 | 4 | default | 1.24 |
| Heuristics | 14 | 4 | default | 1.46 |
| Merged k-maps | 14 | 4 | default | 0.89 |

Table 7–3: Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 4

| Method used to obtain assertions | Total number of assertions | FIFO Size | Engine used | Total CPU time (sec) |
|---|---|---|---|---|
| Past SV Minimized function | 12 | 4 | default | 0.97 |
| Past SV Heuristics | 7 | 4 | default | 1.02 |
| Past SV Merged Karnaugh maps | 6 | 4 | default | 0.92 |

Table 7–3: Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 4

| Method used to obtain assertions | Total number of assertions | FIFO Size | Engine used | Total CPU time (sec) |
|---|---|---|---|---|
| Minimized function | 16 | 8 | default | 22.9 |
| Heuristics | 14 | 8 | default | 22.86 |
| Merged k-maps | 14 | 8 | default | 7.76 |
| Past SV Minimized function | 12 | 8 | default | 12.02 |
| Past SV Heuristics | 7 | 8 | default | 12.68 |
| Past SV Merged Karnaugh maps | 6 | 8 | default | 8.38 |

Table 7–4: Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 8

| Method used to obtain assertions | Total number of assertions | FIFO Size | Engine used | Total CPU time (sec) |
|---|---|---|---|---|
| Minimized function | 16 | 16 | default | 861.38 |
| Heuristics | 14 | 16 | default | 877.36 |
| Merged k-maps | 14 | 16 | default | 250.58 |

Table 7–5: Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 16

| Method used to obtain assertions | Total number of assertions | FIFO Size | Engine used | Total CPU time (sec) |
|---|---|---|---|---|
| Past SV Minimized function | 12 | 16 | default | 460.72 |
| Past SV Heuristics | 7 | 16 | default | 416.08 |
| Past SV Merged Karnaugh maps | 6 | 16 | default | 342.41 |

Table 7–5: Total time to verify equivalent assertion sets in IFV obtained by the methods described in Chapter 6 and FIFO Size set to 16

Table 7–5 shows that, out of all the assertion optimization methods, the merged Karnaugh map provided the best results, that is up to 4 time faster than both the minimized function and heuristic methods and 2 times faster than the past state variables methods. From the test results, the merged Karnaugh map without past state variables, showed better results than the merged Karnaugh map with past state variables.

Also worth noting that the set of heuristics assertions did not provide any gain over the minimized function. While, the addition of past state variables provided significant gains (2x faster) for both minimized function and heuristics.

**CPU Time for Each Assertion**

Tables 7–6 to 7–11 give the CPU time taken by each assertion for the 6 different optimization sets when the FIFO Size was set to 16.

| Source | Assertion | CPU time |
|---|---|---|
| Minimized Functions | {!R & F} \| => {F}; | 0.03s |
| | {!R & W & A} \| => {F}; | 224.37s |
| | {R} \| => {!F}; | 59.44s |
| | {!F & !W} \| => {!F}; | 61.23s |
| | {!F & !A} \| => {!F}; | 62.27s |
| | {!W & E} \| => {E}; | 0.04s |
| | {!W & R & B} \| => {E}; | 102.03s |
| | {W} \| => {!E}; | 84.39s |
| | {!E & !R} \| => {!E}; | 72.50s |
| | {!E & !B} \| => {!E}; | 194.88s |
| | {F & W} \| => {O}; | 0.04s |
| | {!W} \| => {!O}; | 0.01s |
| | {!F} \| => {!O}; | 0.05s |
| | {E & R} \| => {U}; | 0.02s |
| | {!R} \| => {!U}; | 0.02s |
| | {!E} \| => {!U}; | 0.06s |

Table 7–6: CPU Time per Assertions for FIFO_Size=16 with minimized functions

| Source | Assertion | CPU time |
|---|---|---|
| Heuristics | {!R & F} \| => {F};<br>{!R & W & A} \| => {F};<br>{!F & !W} \| => {!F};<br>{!F & !A} \| => {!F};<br><br>{!W & E} \| => {E};<br>{!W & R & B} \| => {E};<br>{!E & !R} \| => {!E};<br>{!E & !B} \| => {!E};<br><br>{!W} \| => {!O};<br>{!R} \| => {!U};<br>{E & R} \| => {U & !F};<br>{!E & R} \| => {!U & !F};<br>{F & W} \| => {O & !E};<br>{!F & W} \| => {!O & !E}; | 0.06s<br>232.55s<br>62.48s<br>60.53s<br><br>0.06s<br>103.59s<br>70.74s<br>194.90s<br><br>0.01s<br>0.00s<br>0.03s<br>53.58s<br>0.07s<br>98.76s |

Table 7–7: CPU Time per Assertions for FIFO_Size=16 with heuristics

| Source | Assertion | CPU time |
|---|---|---|
| | {!R & !F & !E & !W} \| => {!F & !E & !O & !U};<br>{!R & !F & W & !A} \| => {!F & !E & !O & !U};<br>{R & !E & !W & !B} \| => {!F & !E & !O & !U};<br>{R & !F & !E & W} \| => {!F & !E & !O & !U}; | 0.09s<br>0.05s<br>0.05s<br>0.03s |

Table 7–8: CPU Time per Assertions for FIFO_Size=16 with Merged Karnaugh Maps

| **Source** | **Assertion** | **CPU time** |
|---|---|---|
| Merged Karnaugh Maps | {!R & !F & W & A} \| => {F & !E & !O & !U}; | 174.80s |
| | {!R & F & !E & !W} \| => {F & !E & !O & !U}; | 0.04s |
| | {!R & !F & E & !W} \| => {!F & E & !O & !U}; | 0.04s |
| | {R & !E & !W & B} \| => {!F & E & !O & !U}; | 75.21s |
| | {R & F & !E & W} \| => {!F & !E & O & !U}; | 0.06s |
| | {R & !F & E & W} \| => {!F & !E & !O & U}; | 0.03s |
| | {!R & F & E & !W} \| => {F & E & !O & !U}; | 0.01s |
| | {R & F & E & W} \| => {!F & !E & O & U}; | 0.05s |
| | {!R & F & W} \| => {F & !E & O & !U}; | 0.04s |
| | {R & E & !W} \| => {!F & E & !O & U}; | 0.08s |

Table 7–8: CPU Time per Assertions for FIFO_Size=16 with Merged Karnaugh Maps

| **Source** | **Assertion** | **CPU time** |
|---|---|---|
| Past State Variable Minimized Functions | {!R & !A} \| => {$\$stable(F)$}; | 0.07s |
| | {!R & !W} \| => {$\$stable(F)$}; | 0.06s |
| | {!R & W & A} \| => {F}; | 222.33s |
| | {R} \| => {!F}; | 58.37s |
| | {!W & !B} \| => {$\$stable(E)$}; | 0.49s |
| | {!W & !R} \| => {$\$stable(E)$}; | 0.05s |
| | {!W & R & B} \| => {E}; | 98.84s |
| | {W} \| => {!E}; | 80.42s |
| | {W} \| => {$O == \$past(F)$}; | 0.03s |
| | {!W} \| => {!O}; | 0.02s |
| | {R} \| => {$U == \$past(E)$}; | 0.02s |
| | {!R} \| => {!U}; | 0.02s |

Table 7–9: CPU Time per Assertions for FIFO_Size=16 with Past State Variable Minimized Functions

| Source | Assertion | CPU time |
|---|---|---|
| Past State Variable Heuristics | {!R & !W} \| => {$\$stable(F)$ & $\$stable(E)$ & !O & !U}; | 0.14s |
| | {R} \| => {!F & $U == \$past(E)$}; | 40.41s |
| | {W} \| => {!E & $O == \$past(F)$}; | 95.04s |
| | {!R & !A} \| => {$\$stable(F)$ & !U}; | 0.08s |
| | {!R & W & A} \| => {F & !U}; | 165.10s |
| | {!W & !B} \| => {$\$stable(E)$ & !O}; | 0.20s |
| | {R & !W & B} \| => {E & !O}; | 115.11s |

Table 7–10: CPU Time per Assertions for FIFO_Size=16 with Past State Variable Heuristics

| Source | Assertion | CPU time |
|---|---|---|
| Past State Variable Merged Karnaugh Maps | {!R&!W} \| => {$\$stable(F)$ & $\$stable(E)$ & !O & !U}; | 0.13s |
| | {R&W} \| => {!F & !E & $O == \$past(F)$ & $U == \$past(E)$}; | 0.06s |
| | {!R&W&!A} \| => {$\$stable(F)$ & !E & $O == \$past(F)$ & !U}; | 0.08s |
| | {!R&W&A} \| => {F & !E & $O == \$past(F)$ & !U}; | 203.91s |
| | {R&!W&!B} \| => {!F & $\$stable(E)$ & !O & $U == \$past(E)$}; | 0.37s |
| | {R&!W&B} \| => {!F & E & !O & $U == \$past(E)$}; | 137.86s |

Table 7–11: CPU Time per Assertions for FIFO_Size=16 with Past State Variable Merged Karnaugh Maps

What is immediately noticeable from the above table, is that the majority of the assertion took only a fraction of a second while the other half took over 99% of the verification time.

The key points from the Tables 7–6 through 7–11 are that the assertions with A and B in them, in the minimized function and heuristics, took the most time to prove while the assertions for overflow and underflow were verified instantly. Noticeably, the assertions for E (empty) and F (full) took a significant amount of time.

On the other hand, in the merged Karnaugh map cases, the assertions that took the most significant amount of time contain A and B (A refers to the condition when the write address is equal to the read address minus 1, and B refers to the condition when the read address is equal to the write address minus 1).

**Interpretation of Results**

The first thing to note is that increasing the FIFO size only affects certain signals. A and B depend directly on the FIFO Size (`A = write_addr + 1 = read_addr` and the length of `write_addr is [log(FIFO_Size)-1:0]` ).

The result looked at first as unexpected, however, an attentive reflection reveals that in a formal verification setting, adding more dependencies to an assertion reduces the number of states that the formal verification tool needs to explore in order to prove whether the assertion is true or false.

Therefor, the more dependencies added to an assertion, in a formal setting, the faster the assertion will be verified. This also explains why the merged Karnaugh maps did exceptionally well while the heuristics test did poorly in comparison.

# CHAPTER 8
# Conclusions and Future Work

## 8.1 Summary

The objective of this thesis was to provide answers to the questions posed in the introduction, "when have sufficient assertions been written?", "how can it be determined whether additional non-redundant assertions can be added to an assertion set?" and "how to optimize the existing set of assertion for both formal and dynamic verification?"

As discussed in Chapter 3.4, a divide and conquer approach is proposed to tackle the question "when have sufficient assertions been written". To accomplish this, the different types of assertions had to be identified based on their purpose from the designer's and verification engineer's point of view. This will allow the verifiers to simplify the task of writing sufficient assertions on a type by type basis.

The optimization techniques described in Chapter 6 are mainly applicable to behavioral assertions. The designers are encouraged to write the function and inverse function for all the important control logic. The Boolean function method is a methodology that guarantees that all desirable behavioral assertions have been written and are non-redundant.

Once the set of behavioral assertions were obtained, logic synthesis optimization techniques were then carried out on the set of non-redundant assertions, with the

purpose of taking advantage of the common features among separate assertions. Merging Karnaugh maps and the use of heuristics (similar to EXPRESSO) were applied to optimize such a set of assertions.

This thesis also addresses the use of invariant assertions, it was shown that certain invariant assertions can be automatically generated from the Boolean functions. Such invariant assertions can be regarded as redundant to behavioral assertions and may be omitted to speed up the verification process.

Once the various sets of assertions was obtained, one then proceeds to compare the efficiency of these assertions in a formal and dynamic verification setting such as IFV. It was shown that in a formal setting, merging Karnaugh maps allowed IFV to prove the assertion set 4 times faster than the initial non-optimized set of non-redundant assertions, while Karnaugh maps did not provide any gain.

## 8.2 Future Work

The tests were conducted in a formal verification environment and the results obtained in Chapter 7 can only be analyzed in a formal setting. Future work could investigate how well heuristics, merged Karnaugh maps and past state variables work in a dynamic simulation setting. The lack of significant gains with the heuristics set of assertions was particularly disappointing from my point of view, but it is still possible that the potential of heuristics could be found in a dynamic verification setting.

In Chapter 6, the stated goal of optimizing assertions was to minimize the assertion count and the literal count in each assertion. It turns out that in a formal

setting, from analyzing the results in Chapter 7, that when the literal count increases, the formal tool can verify the assertion faster. This may be the reason why merged Karnaugh maps produced the best results.

Future work would study the effects of the optimization methods in a dynamic simulation setting. The use of to state variables must also be studied in dynamic simulations to determine if the use of $\$past()$ and $\$stable()$ functions slow down the speed of verification. Many more tests on a wider variety of circuits would also be useful in order to cover a wider set of designs and measure the usefulness of the proposed optimizations in various settings.

Although the use of the $\$past()$ and $\$stable()$ PSL functions was investigated, PSL contains many more functions and operators and it would be useful to investigate if other functions can provide further gains.

In terms of optimization, the thesis focused on optimizing behavioral assertions. Other types of assertions can still be investigated. For example, optimizing temporal assertions was conducted on the grant signal in Section 4.3 where instead of requiring 3N assertions to verify the set of grant assertions, only 2 assertions were needed.

## Appendix A: Heuristics Applied to FEOU

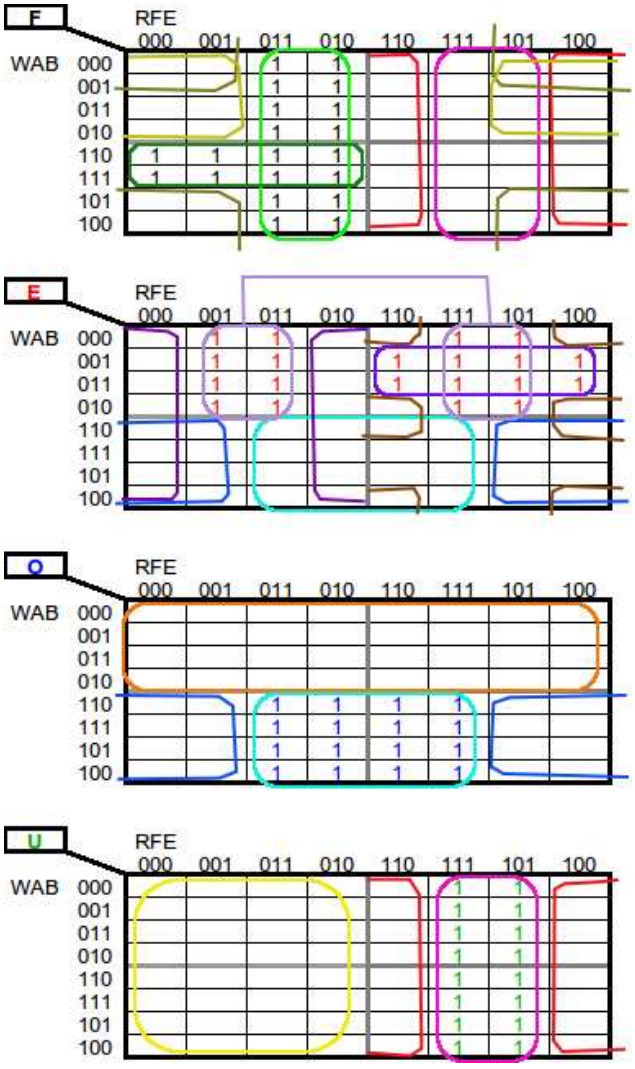



Figure 8–1: Heuristics applied to FEOU

## Appendix B: FEOU Assertions Obtained from the Various Methods

| Assertions obtained from the minimized functions in table 6–4 | Assertions obtained from the merged Karnaugh map in figure 6–10 |
|---|---|
| {!R & F} \| => {F};<br>{!R & W & A} \| => {F};<br>{R} \| => {!F};<br>{!F & !W} \| => {!F};<br>{!F & !A} \| => {!F};<br><br>{!W & E} \| => {E};<br>{!W & R & B} \| => {E};<br>{W} \| => {!E};<br>{!E & !R} \| => {!E};<br>{!E & !B} \| => {!E};<br><br>{F & W} \| => {O};<br>{!W} \| => {!O};<br>{!F} \| => {!O};<br><br>{E & R} \| => {U};<br>{!R} \| => {!U};<br>{!E} \| => {!U}; | {!R & !F & !E & !W} \| => {!F & !E & !O & !U};<br>{!R & !F & W & !A} \| => {!F & !E & !O & !U};<br>{R & !E & !W & !B} \| => {!F & !E & !O & !U};<br>{R & !F & !E & W} \| => {!F & !E & !O & !U};<br><br>{!R & !F & W & A} \| => {F & !E & !O & !U};<br>{!R & F & !E & !W} \| => {F & !E & !O & !U};<br><br>{!R & !F & E & !W} \| => {!F & E & !O & !U};<br>{R & !E & !W & B} \| => {!F & E & !O & !U};<br><br>{R & F & !E & W} \| => {!F & !E & O & !U};<br>{R & !F & E & W} \| => {!F & !E & !O & U};<br><br>{!R & F & E & !W} \| => {F & E & !O & !U};<br>{R & F & E & W} \| => {!F & !E & O & U};<br>{!R & F & W} \| => {F & !E & O & !U};<br>{R & E & !W} \| => {!F & E & !O & U}; |
| Total of 16 Assertions | Total of 14 Assertions |

Table 8–2: FEOU Assertions Table 1/5

| Assertions obtained from the heuristic method applied in figure 8–1 in Appendix A | Assertions obtained from the minimized functions with state variables in figure 6–15 |
|---|---|
| {!R & F} \| => {F};<br>{!R & W & A} \| => {F};<br>{!F & !W} \| => {!F};<br>{!F & !A} \| => {!F};<br><br>{!W & E} \| => {E};<br>{!W & R & B} \| => {E};<br>{!E & !R} \| => {!E};<br>{!E & !B} \| => {!E};<br><br>{!W} \| => {!O};<br>{!R} \| => {!U};<br><br>{E & R} \| => {U & !F};<br>{!E & R} \| => {!U & !F};<br><br>{F & W} \| => {O & !E};<br>{!F & W} \| => {!O & !E}; | {!R & !A} \| => {$\$stable(F)$};<br>{!R & !W} \| => {$\$stable(F)$};<br>{!R & W & A} \| => {F};<br>{R} \| => {!F};<br><br>{!W & !B} \| => {$\$stable(E)$};<br>{!W & !R} \| => {$\$stable(E)$};<br>{!W & R & B} \| => {E};<br>{W} \| => {!E};<br><br>{W} \| => {$O == \$past(F)$};<br>{!W} \| => {!O};<br><br>{R} \| => {$U == \$past(E)$};<br>{!R} \| => {!U}; |
| Total of 14 Assertions | Total of 12 Assertions |

Table 8–3: FEOU Assertions Table 2/5

| Assertions obtained from the merged Karnaugh map with state variables in figure 6–16 |
|---|
| {!R & !W} \| => {$\$stable(F)$ & $\$stable(E)$ & !O & !U};<br>{R & W} \| => {!F & !E & $O == \$past(F)$ & $U == \$past(E)$}; |
| {!R & W & !A} \| => {$\$stable(F)$ & !E & $O == \$past(F)$ & !U};<br>{!R & W & A} \| => {F & !E & $O == \$past(F)$ & !U}; |
| {R & !W & !B} \| => {!F & $\$stable(E)$ & !O & $U == \$past(E)$};<br>{R & !W & B} \| => {!F & E & !O & $U == \$past(E)$}; |
| Total of 6 Assertions |

Table 8–4: FEOU Assertions Table 3/5

| Assertions obtained from the heuristic method with state variables applied in figure 6–18 |
|---|
| {!R & !W} \| => {$\$stable(F)$ & $\$stable(E)$ & !O & !U};<br>{R} \| => {!F & $U == \$past(E)$};<br>{W} \| => {!E & $O == \$past(F)$};<br><br>{!R & !A} \| => {$\$stable(F)$ & !U};<br>{!R & W & A} \| => {F & !U};<br><br>{!W & !B} \| => {$\$stable(E)$ & !O};<br>{R & !W & B} \| => {E & !O}; |
| Total of 7 Assertions |

Table 8–5: FEOU Assertions Table 4/5

| Applying temporal operators on each method |
|---|
| When temporal operators were applied on the minimized assertions in section 6.2.4 the result was the reduction of the number of assertions by 2.<br><br>The 2 assertions that were removed from the list in table 8–2 are:<br>{!F} \| => {!O}<br>{!E} \| => {!U}<br><br>and the modified assertions became:<br>{R} \| => {!F; !O}<br>{!F & !W} \| => {!F; !O}<br>{!F & !A} \| => {!F; !O}<br>{W} \| => {!E; !U}<br>{!E & !R} \| => {!E; !U}<br>{!E & !B} \| => {!E; !U}<br><br>Attempting to apply this technique on all of the remaining cases did not succeed because none of the conditions described in table 6–12 concerning temporal operators were met. |

Table 8–6: FEOU Assertions Table 5/5

References


[1] acme labs. *Safety and Liveness Properties.*

[2] S. Aggarwal, S. Mittal, and S.B. Garg. Assertion based verification. *Soft computing*, page 415, 2005.

[3] Sanghavi Alok. *What is formal verification?*

[4] A. Banerjee, K. Datta, and P. Dasgupta. Checkspec: A tool for consistency and coverage analysis of assertion specifications. *Automated Technology for Verification and Analysis*, pages 228–233, 2008.

[5] A. Banerjee, B. Pal, C. Kamarapu, P. Dasgupta, PP Chakrabarti, and M. Jha. Assertion based verification: have i written enough properties? In *India Annual Conference, 2004. Proceedings of the IEEE INDICON 2004. First*, pages 363–367. IEEE, 2004.

[6] Prabal Bhattacharya and Don O'Riordan. *Assertion-based verification in mixed-signal design*, 10/17/2011.

[7] J. Bormann, A. Fedeli, R. Frank, and K. Winkelmann. Combined static and dynamic verification. *Prosyd D*, 3(1), 2005.

[8] M. Boulé and Z. Zilic. Incorporating efficient assertion checkers into hardware emulation. *IEEE Computer Society*, 2005.

[9] M. Boule and Z. Zilic. Efficient automata-based assertion-checker synthesis of seres for hardware emulation. In *2007 Asia and South Pacific Design Automation Conference*, pages 324–329. IEEE, 2007.

[10] M. Boulé and Z. Zilic. Automata-based assertion-checker synthesis of psl properties. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 13(1):4, 2008.

[11] M. Boulé and Z. Zilic. *Generating hardware assertion checkers: for hardware verification, emulation, post-fabrication debugging and on-line monitoring*. Springer Verlag, 2008.

[12] Cadence. *SystemVerilog Assertions (SVA) EZ-Start Guide, August 2006*.

[13] R. Cavada, A. Cimatti, C.A. Jochim, G. Keighren, E. Olivetti, M. Pistore, M. Roveri, and A. Tchaltsev. Nusmv 2.4 user manual. *ITCIRST*, 1998.

[14] E. Cerny, S. Dudani, J. Havlicek, and D. Korchemny. *The power of assertions in systemverilog*. Springer Verlag, 2010.

[15] A. Cimatti, E. Clarke, E. Giunchiglia, F. Giunchiglia, M. Pistore, M. Roveri, R. Sebastiani, and A. Tacchella. Nusmv 2: An opensource tool for symbolic model checking. In *Computer Aided Verification*, pages 241–268. Springer, 2002.

[16] A. Cimatti, E. Clarke, F. Giunchiglia, and M. Roveri. Nusmv: A new symbolic model verifier. In *Computer Aided Verification*, pages 682–682. Springer, 1999.

[17] C. Coelho and H. Foster. Assertion-based verification. *Advanced Formal Verification*, pages 167–204, 2004.

[18] B. Cohen, S. Venkataramanan, and A. Kumari. *Using PSL/Sugar for formal and dynamic verification: guide to property specification language for assertion-based verification*. vhdlcohen publishing, 2004.

[19] B. Cohen, S. Venkataramanan, and A. Kumari. *System Verilog assertions handbook*. vhdlcohen publishing, 2005.

[20] A. Dahan, D. Geist, L. Gluhovsky, D. Pidan, G. Shapir, Y. Wolfsthal, L. Benalycherif, R. Kamidem, and Y. Lahbib. Combining system level modeling with assertion based verification. In *Quality of Electronic Design, 2005. ISQED 2005. Sixth International Symposium on*, pages 310–315. IEEE, 2005.

[21] G. De Micheli et al. *Synthesis and optimization of digital circuits*, volume 107. McGraw-Hill New York, 1994.

[22] Karmel Electronic Design. *Assertion Libraries Development*, last accessed on 03/17/2012.

[23] Doulos. *SystemVerilog Assertions Tutorial*.

[24] R. Drechsler. *Advanced formal verification*. Springer, 2004.

[25] B. Elkarablieh, I. Garcia, Y.L. Suen, and S. Khurshid. Assertion-based repair of complex data structures. In *Proceedings of the twenty-second IEEE/ACM international conference on Automated software engineering*, pages 64–73. ACM, 2007.

[26] J. Michelson F. Haque. *Practical Approaches to Deployment of SystemVerilog Assertions*, 4/3/2007.

[27] H. Foster. Applied assertion-based verification: An industry perspective. *Foundations and Trends in Electronic Design Automation*, 3(1):1–95, 2009.

[28] H. Foster, A. Krolnik, and D. Lacey. *Assertion-based design*. Springer, 2004.

[29] H. Foster, K. Larsen, and M. Turpin. Introduction to the new accellera open verification library. In *DVCon06: Proceedings of the Design and Verification Conference and exhibition*, 2006.

[30] Harry D. Foster. *Accellera Verilog++ Assertion Extension Requirements Proposal*.

[31] A. Ghofrani, F. Javaheri, S. Safari, and Z. Navabi. Automatic insertion of efficient assertions in combinational gate level circuits using particle swarm optimization.

[32] Richard Goering. *DVClub*, April 26, 2011.

[33] G.D. Hachtel and F. Somenzi. *Logic synthesis and verification algorithms*. Springer Verlag, 2006.

[34] S. Hassoun and T. Sasao. *Logic synthesis and verification*, volume 654. Springer Netherlands, 2002.

[35] A. Hekmatpour and A. Salehi. Systems, methods, and media for block-based assertion generation, qualification and analysis, December 25 2007. US Patent 7,313,772.

[36] S. Hessabi, A.M. Gharehbaghi, B.H. Yaran, and M. Goudarzi. Integrating assertion-based verification into system-level synthesis methodology. In *Microelectronics, 2004. ICM 2004 Proceedings. The 16th International Conference on*, pages 232–235. IEEE, 2004.

[37] M. Joshi. Techniques for maximizing assertion-based verification throughput in acceleration and emulation. *Cadence User Forums, DOI= http://www. cdnusers. org/Portals/0/cdnlive/na2006/PNP/PNP_403/403_paper. pdf*, 2006.

[38] S. Katz, O. Grumberg, and D. Geist. Have i written enough properties? - a method of comparison between specification and implementation. *Correct hardware design and verification methods*, pages 704–704, 1999.

[39] Y. Lu. A unified verification flow using assertion synthesis technology.

[40] Kramer Magee. *Chapter 7 - Safety & Liveness Properties.*

[41] P. Marriot. *The What, When, and How Much of functional coverage*, September 1, 2006.

[42] A. OneSpin Solutions Nordstrom. *How do I know I have written enough assertions.*

[43] M. Riazati, S. Mohammadi, and Z. Navabi. Non-overlapping set of efficient assertions. In *Norchip Conference, 2006. 24th*, pages 201–204. IEEE, 2006.

[44] A.P. Sistla. Safety, liveness and fairness in temporal logic. *Formal Aspects of Computing*, 6(5):495–511, 1994.

[45] Cadence Design Systems. *Assertions Based Verification Workshops.*

[46] Cadence Design Systems. *Developing Properties for Functional Verification, Incisive Formal Verification.*

[47] Y. Tao. An introduction to assertion-based verification. In *ASIC, ASICON'09. IEEE 8th International Conference on*, pages 1318–1323. IEEE, 2009.

[48] J.G. Tong, M. Boulé, and Z. Zilic. Airwolf-tg: A test generator for assertion-based dynamic verification. In *High Level Design Validation and Test Workshop, 2009. HLDVT 2009. IEEE International*, pages 106–113. IEEE, 2009.

[49] J.G. Tong, M. Boulé, and Z. Zilic. Defining and providing coverage for assertion-based dynamic verification. *Journal of Electronic Testing*, 26(2):211–225, 2010.

[50] J.G. Tong, D. Sarraf, M. Boule, and Z. Zilic. Generating compact assertions for control-based logic signals. In *Circuits and Systems (MWSCAS), 2011 IEEE 54th International Midwest Symposium on*, pages 1–4. IEEE, 2011.

[51] Project VeriPage. *SystemVerilog Assertion*.

[52] S. Vijayaraghavan and M. Ramanathan. *A practical guide for System Verilog Assertions*. Springer Verlag, 2005.

[53] vmmcentral.org. *Introduction to Design Verification with VMM*.

[54] wikipedia. *Verification and validation*.

[55] T. Williams. *Covered - Verilog Code Coverage Analyzer*.

[56] P. Yeung and K. Larsen. Practical assertion-based formal verification for soc designs. In *System-on-Chip, Proceedings. 2005 International Symposium on*, pages 58–61. IEEE, 2005.

[57] Z. Zilic. *Course Notes for ECSE 530, Logic Synthesis*, Winter 2011.

[58] Inc Zocalo Tech. *Enabling Assertion Based Verification*.